Gymnase de Nyon

Travail de Maturité 2024

How to Create and Optimize a Microprocessor-Based System for a Video Game Like Tetris

ADRIAN VLASOV

Class: 3M02

Main Part

Pages 1-33

Supervisor: Mr. Thomas Morel

Expert: Mr. René Beuchat

Machine Translation

This document is an English machine translation of the original French text, produced by **Zhipu AI GLM-5.2** in July 2026.

The original French version remains the authoritative reference.

February 2024

Contents

Table of Contents **1**

1 Introduction **2**

2 Abstract **3**

3 Approach and Choices **4**
3.1 Background and choice of approach . . . 4
3.2 Flags . . . 5
3.3 Interrupts . . . 6
3.4 Memory and data bus . . . 6
3.5 The use of an FPGA . . . 7

4 Operation **8**
4.1 Glossary . . . 8
4.2 Preliminary concepts . . . 8
4.3 Implementations . . . 8
4.4 Details of processor operation . . . 9
4.5 Parts of the processor . . . 9
4.6 Prototype . . . 12
4.7 A few details . . . 14

5 Programming the Assembler Compiler **15**
5.1 Introduction to assembly . . . 15
5.2 How the compiler works . . . 16

6 Programming Tetris **17**
6.1 Programming in C . . . 17
6.2 Transcription of the algorithm into assembly . . . 18
6.3 Memory organization . . . 22

7 Sound Implementation **22**
7.1 Physical operation . . . 22
7.2 How a synthesized sound works . . . 23
7.3 Computation of the sound . . . 23
7.4 Playing a sound . . . 24

8 Results obtained **24**
8.1 Recap . . . 24
8.2 Technical details . . . 25
8.3 Comparison with other processors . . . 26
8.4 Analysis of the results . . . 27

9 Conclusion **28**
9.1 Recap . . . 28
9.2 Critique . . . 28
9.3 Going further . . . 29

10 Acknowledgements **29**

11 Bibliography **30**

12 Appendices **34**
12.1 Implementation details . . . 34
12.2 Problems encountered . . . 34
12.3 Format of the instruction set . . . 34
12.4 Bit fields . . . 35
12.5 Instruction set . . . 40
12.6 Interview . . . 44
12.7 Source Code and Files . . . 45

1 Introduction

Moore's Law[1] states that the number of transistors in an integrated circuit doubles every two years with no significant increase in production costs. This is what Gordon Moore (Figure 1), co-founder of Intel, predicted in 1975. This observation has even remained valid up to the present day. But the future is inevitable; we cannot possibly continue in this direction. Indeed, the size of transistors is bounded by physical constraints, such as the size of atoms,[2] or by the way transistors are manufactured,[3] and therefore Moore's Law cannot remain valid forever.



Figure 1: Gordon Moore at Intel

We can therefore no longer rely on the ability of processors to run our unoptimized programs ever faster than before. We will therefore have to prepare to optimize[4] applications.

From an environmental point of view, consumers, by always wanting the newest device, throw away their old devices which they find too slow, even though these could still be used if the programs were more optimized.[5] Moreover, a large number of multinational companies fuel this phenomenon by dropping support for older devices. A prime example of this is Microsoft, with the very popular Windows 11 operating system, which dropped update support for computers a little over fifteen years old, which would force consumers to buy a new computer. Furthermore, unoptimized applications consume more electricity to run[6],[7] which has a negative impact on the environment due to the waste of electricity.

From a financial point of view, optimizing applications for one or two hundred different devices will cost more than increasing the processor speed of all machines.[8] To illustrate this point, let us look at the case of Android, the widely used operating system. The majority of phone manufacturers use Android (Samsung, for example[9]), and there is a wide variety of Android smartphones on the market. Optimizing an application for each device will therefore cost more than increasing the processor speed of all machines. This is why developers do not want to optimize their programs for every phone.[10] To cope with the problem of software slowness, the manufacturer of the device in question equips its products with a processor that is even more powerful than the previous generation. For example, Samsung mentions that phones are getting faster[11] and that using a new processor will help reduce sluggishness.

In the past, programs were more optimized and programmers tried to make good use of all the resources provided by the manufacturer of the device in question,[12] but now programmers no longer need to worry about processor speeds because these have become so fast that most applications run very well without optimization.

In this Travail de Maturité, we will build a system from scratch to show that by optimizing programs, one can have a very fast system with a very simple processor. In order to validate the operation of this processor, we will run the game Tetris[13] on it. Then, we will compare our implementation to those of a

[1] Intel Corporation. *Moore's Law - The past, present and future of Gordon Moore's golden rule for the semiconductor industry. [En Ligne].* Page consultée le 17 Mars 2024. URL: https://www.intel.com/content/www/us/en/newsroom/resources/moores-law.html.

[2] NIDEC Group. *Calculating Infinity: The Paradox Of Moore's Law [En Ligne].* Page consultée le 26 Mars 2024. URL: https://acim.nidec.com/drives/control-techniques/news-and-media/blog/insights/calculating-infinity-the-paradox-of-moores-law.

[3] Arnab Hazari The Conversation Media Group Ltd. *The future of electronics is light.* Page consultée le 27 Mars 2024. URL: https://theconversation.com/the-future-of-electronics-is-light-68903.

[4] In this context, the term "optimize" refers to improving the efficiency of applications in order to increase their performance while using the same system resources as before.

[5] PCWorld - IDG Communications, Inc. *Shopping for an older CPU? Don't go past this—or you'll regret it.* Page consultée le 27 Mars 2024. URL: https://www.pcworld.com/article/539090/shopping-for-an-older-cpu-dont-go-past-this-or-youll-regret-it.html.

[6] David Cassel. *Which Programming Languages Use the Least Electricity?* Page consultée le 27 Mars 2024. URL: https://thenewstack.io/which-programming-languages-use-the-least-electricity/.

[7] Muhammad Umair Khan et al. *Measuring power consumption in mobile devices for energy sustainable app development: A comparative study and challenges.* Page consultée le 27 Mars 2024. URL: https://www.sciencedirect.com/science/article/abs/pii/S2210537921000780.

[8] Ayush Jalan www.makeuseof.com. *Why Do So Many Developers Launch Their Apps on iOS Before Android?* Page consultée le 27 Mars 2024. URL: https://www.makeuseof.com/why-are-cross-platform-apps-released-on-ios-first/.

[9] SAMSUNG. *One UI Beta Program - Experience the latest One UI with Android 14.* Page consultée le 27 Mars 2024. URL: https://developer.samsung.com/one-ui-beta.

[10] www.makeuseof.com, *Why Do So Many Developers Launch Their Apps on iOS Before Android?*, op. cit.

[11] SAMSUNG. *Samsung Galaxy S24 Ultra.* Page consultée le 27 Mars 2024. URL: https://www.samsung.com/us/smartphones/galaxy-s24-ultra/.

[12] Patrik Tennberg. *The lost art of code optimization.* Page consultée le 27 Mars 2024. URL: https://www.linkedin.com/pulse/lost-art-optimization-patrik-tennberg.

[13] Joel Lee www.makeuseof.com. *30 Years Of Tetris: A History Of The World's Most Beloved Puzzle Game.* Page consultée le 27 Mars 2024. URL: https://www.makeuseof.com/tag/30-years-tetris-history-worlds-beloved-puzzle-game/.

few modern computers in terms of speed, using parameters such as the clock speed and the number of instructions processed per clock cycle, but also in terms of electricity consumption.

To carry out this ambitious work, several issues will need to be settled. How should the processor be organized, that is, how to choose an efficient architecture for this implementation? What compromises should be made to have a processor that is very simple yet fast at the same time? To reach an answer to these questions, we will draw up a minimal list of instructions for our processor, which will lead to the different parts to be implemented (for example ALU,[14] external connections, etc.). To test Tetris on our system, we will connect peripherals to our processor such as a screen, a few buttons, and a speaker. Finally, using this, we will add sound to our game.

In a final part, we will compare our implementation to those of a few computers in use today.

2 Abstract

We asked ourselves whether a processor, even with a clock frequency as low as $40MHz$ (this frequency is slow nowadays, but many processors ran at 8MHz in the 1980s), could run a game like Tetris, because we had written it in assembly. This turned out to be possible, so we created a complete processor capable of running Tetris with sound. To achieve this, we implemented a 64-bit processor and an assembler compiler so that we could program the game Tetris in assembly. We also implemented sound for this game.

Our system uses the CMOD-A7 printed circuit board from Digilent, which contains an Artix-7 FPGA. We used the Vivado software to program the FPGA, since it was manufactured by Xilinx.

Being implemented on an FPGA, our processor has a clock frequency of $40MHz$ and is able to run Tetris without any problem. It has 8 GPIO inputs and outputs, the last 2 of which are PWM outputs for sound. These PWM outputs are the inverse of each other. The resolution of these PWM outputs is 8 bits. Our processor has 16 registers of 64 bits.

We also compared our processor with other processors, such as the *ESP32* or the *STR71xF*. The number of instructions per clock cycle of our processor (MIPS/MHz) turned out to be higher (1.0) than that of the *STR71xF* (0.9) and the *ESP32* (0.6). We also measured the power consumption of our processor, which is $0.311W$. We therefore succeeded in creating a very simple yet complete processor, capable of running Tetris with sound, with very low power consumption. We achieved our goal.

[14] Lenovo. *ALU: Arithmetic Logic Unit Key Components & Logic Gates*. Page consultée le 7 Avril 2024. URL: https://www.lenovo.com/ph/en/glossary/arithmetic-logic-unit/.

3 Approach and Choices

To design a processor, one must first know the goal. Some make processors that can quickly compute functions such as sine and cosine for graphics cards (Nvidia, for example); others invent an architecture for supercomputers (such as PowerPC). Our goal here is to create the simplest possible processor that should run Tetris, and then to compare its speed with today's mainstream processors (such as the ATmega328P[15] of the Arduino,[16] for example).

3.1 Background and choice of approach

Note that, historically, processors were first drawn by hand on Mylar (a type of thin polyester sheet), then transferred onto "Rubylith" (Figure 2). Rubylith is a two-layer material, whose base layer is transparent Mylar covered with a thin layer of a material resembling red cellophane. The first chips manufactured at Intel used a machine called a "Coordinatograph" to help cut the red layer (called the ruby layer). However, the positions and lengths of the various elements of the processor drawn on the Mylar had to be transferred by hand to the coordinatograph, which was later replaced by another machine using knives instead of pens for greater efficiency and precision.[17] Nowadays, Hardware Description Languages (HDLs) exist to describe low-level computer hardware (i.e., a chip, a processor). The most widely used languages are Verilog and VHDL. We chose to use Verilog because it is more concise and because we are constrained by the short nine-month time window.[18]



Figure 2: An engineer drawing a circuit on Rubylith

Since we are aiming to build the simplest possible processor due to the time limit, we chose a RISC (Reduced Instruction Set Computer) type of architecture. We therefore did not opt for a larger number of more complex instructions (i.e., a CISC, Complex Instruction Set Computer, an architecture with an extended instruction set, thus with more complex instructions) because of our decision to make a simple processor, since, generally, the more instructions there are, the more room the processor takes up on the integrated circuit. Thus, we decided to limit ourselves to a maximum of 64 instructions.

We also considered building our processor in 64 bits to see how difficult it would be to make a processor like those in our current computers (x86,[19] ARM[20]). For simplicity, the instruction code of our processor (the opcode) is encoded in 64 bits, of which the 6 most significant bits (6 bits, because 64 instructions, and $6 = log_2(64)$) designate the number of the chosen instruction. The length of our instruction code is always 64 bits and is therefore non-variable, for a lighter architecture. We chose to equip our processor with 16 general-purpose registers, even though there are processors with more or fewer registers (such as

[15] Atmel Corporation. *8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash*. Page consultée le 26 Mai 2024. URL: http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.

[16] Arduino S.r.l. *Arduino Uno Rev3*. Page consultée le 26 Mai 2024. URL: https://store.arduino.cc/products/arduino-uno-rev3.

[17] Andrew M. Volk, Peter A. Stoll, and Paul Metrovich. *Recollections of Early Chip Development at Intel*. Page consultée le 26 Mai 2024. URL: https://deramp.com/downloads/mfe_archive/050-Component%20Specifications/Intel/Recollections%20of%20Early%20Chip%20Dev.pdf.

[18] Diferr.com. *10 Differences Between verilog and vhdl*. Page consultée le 26 Mai 2024. URL: https://diferr.com/difference-between-verilog-and-vhdl/.

[19] Microsoft. *x86 Architecture*. Page consultée le 1er Juin 2024. URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/x86-architecture.

[20] Arm Limited. *CPU Architecture*. Page consultée le 1er Juin 2024. URL: https://www.arm.com/architecture/cpu.

MIPS[21] with 32 registers, or x86 with 8 registers[22]). We also decided not to implement special registers (such as the stack pointer, which indicates the top of the stack and is used to save addresses in recursive functions) in order to simplify the processor.

To represent negative numbers, we used two's complement representation (to convert a number x into a negative number, it is enough to invert each binary digit of this number, and then to add 1 to it). This representation allows us to perform the same operations on a number whether it is negative or not. We decided not to implement floating-point numbers,[23] because for a simple processor we do not need to compute them natively, and such an implementation would require a floating-point unit to compute everything. The computations performed by this unit never take a single cycle (a cycle means that after one clock edge the result will be available for use; when more than one cycle is needed for an instruction, this is called pipelining, which means that at each clock edge the processor computes a small part of the instruction and finally produces the final result[24]), but generally more than 2 cycles (for example, 3 cycles for floating-point multiplication and at least 19 cycles for floating-point division on the 32-bit Intel Pentium processor[25]) in order not to slow down the processor. We also decided to implement neither multiplication nor division in our ALU (Arithmetic Logic Unit, essentially the processor's calculator) so as not to create a multi-cycle pipeline, which would take us a long time to implement, and so that the ALU computes everything in a single cycle, which would simplify the coding.

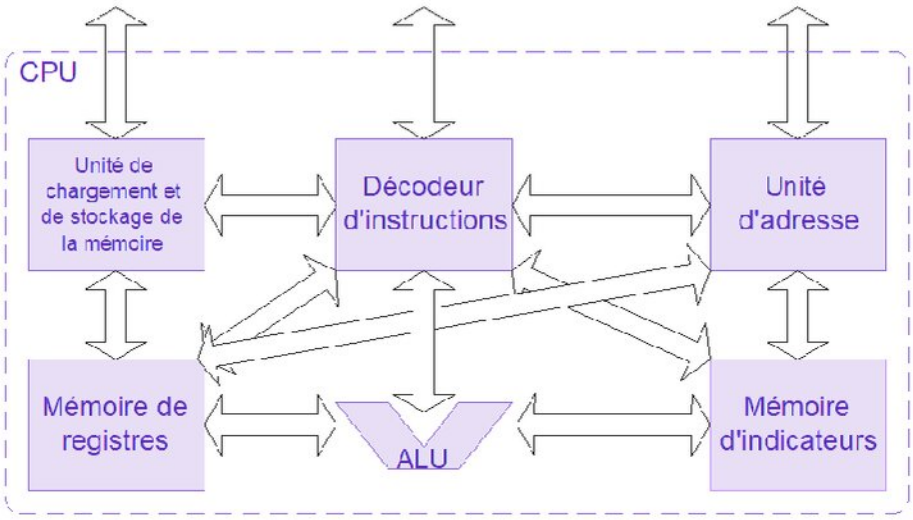


Figure 3: The units typically found in a processor

3.2 Flags

We chose to equip our processor with 4 flags: *Zero*, *Negative*, *Carry*, and *Overflow*.

The Zero flag is set by the ALU when the result of its last operation is zero.

The Negative flag is set by the ALU when the result of the last operation is negative. To compute this, we implemented a second 65-bit operation result specifically for computing this flag. To determine this flag, we check whether the 65ème bit or the 64ème bit of this result is positive. This holds true because any negative number represented using two's complement has its most significant bit set to 1. By adding a negative number to a positive number, the most significant bit remains 1. But if we were to add a negative number to a negative number, the 64ème bit would become 0. This is why we also chose to check the 65ème bit (Figures 4 and 5).

The Carry flag is set when the result of the operation overflows 64 bits, that is, when there has been a carry. To compute it, we simply look at the 65ème bit of the result.

[21] Crystal Chen, Greg Novick, and Kirk Shimano. *MIPS*. Page consultée le 10 Juillet 2024. URL: https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/mips/index.html.

[22] Microsoft, *x86 Architecture*, op. cit.

[23] Jean-Michel Muller et al. *Handbook of Floating-Point Arithmetic*. Page consultée le 27 Mai 2024. URL: https://ieeemilestones.ethw.org/w/images/5/54/Handbook_Floating_Point_Chapter_3.pdf.

[24] Crystal Chen, Greg Novick, and Kirk Shimano. *How Pipelining Works*. Page consultée le 27 Mai 2024. URL: https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/pipelining/index.html.

[25] Agner Fog. *Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD, and VIA CPUs*. Page consultée le 27 Mai 2024. URL: https://www.agner.org/optimize/instruction_tables.pdf.

	63		0
	1111	···	0110
	1111	···	1010
1	1111	···	0000

Figure 4: A negative number added to another negative number

	63		0
	1111	···	0110
	0000	···	1000
0	1111	···	1110

Figure 5: A negative number added to a positive number

The Overflow flag is set when the result of the last operation cannot be used because it has overflowed the positive number space and become negative in the two's complement representation. To compute it, we perform an exclusive OR between the 65ème and the 64ème bit of the result. If the result has both its 65ème and its 64ème bit at 0, then it is positive. If the result has both its 65ème and its 64ème bit at 1, then it is negative. If they are not equal, then something has gone wrong. This flag is therefore set to 1 when the result of the operation is unreliable and thus when we cannot make use of it.

3.3 Interrupts

We also decided to set interrupts aside, even though almost all other mainstream processors have them (for example ARM,[26] Atmega128,[27] Zilog Z80[28]), because of a matter of time. Interrupts are generally inputs of the processor which, by means of a signal, cause it to momentarily stop and go execute a predefined program. In general, the execution of instructions jumps to a line predefined in advance by the programmer after having saved the context (the flags, in order to be able to restore them afterwards). While the interrupt program is executing, all other interrupts are blocked. The program ends with an instruction telling the processor to return to the line from which it abruptly departed and to reload the flag context, that is, to change the flags. At that point, interrupts stop being blocked and this process can repeat once again.

3.4 Memory and data bus

For memory, we decided to implement a module used to switch between the different memories of the processor (such as the ROM containing the program, the RAM, and the inputs or outputs of the processor). By setting the 32ème bit, the programmer can access the various inputs and outputs of the processor. This bit was chosen arbitrarily, while making sure that the very first addresses remain free for the ROM, because at startup our processor will begin by reading the program from address 0. For sound, we equipped the processor with PWM outputs (Pulse Width Modulation, which generates square-wave signals of variable length; these signals are integrated by the speaker as a function of time to produce an analog signal).

We placed the program memory (the one containing the instructions) and the RAM in the same memory block, so that we could run programs directly from RAM rather than from ROM. When the program is in ROM, i.e., directly on the chip, it is implemented using a multiplexer based on the instruction address. This uses lookup tables on the chip. So the larger the program, the more lookup tables there are, the more room the processor takes up on the chip, and the slower it is. By placing the program in RAM, we can not only get rid of this problem but also place the program's data (such as the Tetris game pieces, or a font for writing to the screen) in the same location as the program.

We can therefore come to the conclusion that the architecture of our processor is a Harvard-type architecture (Figure 6), since the data and instruction buses are separate.

[26] Ahmed Fathy Mohammed Abdelrazek. *Exception and Interrupt Handling in ARM*. Page consultée le 27 Mai 2024. URL: http://classweb.ece.umd.edu/enee447.S2016/ARM-Documentation/ARM-Interrupts-3.pdf.

[27] Atmel Corporation. *8-bit Atmel Microcontroller with 128KBytes In-System Programmable Flash*. Page consultée le 27 Mai 2024. URL: https://ww1.microchip.com/downloads/en/DeviceDoc/doc2467.pdf.

[28] Zilog, Inc. *Z80 CPU*. Page consultée le 26 Mai 2024. URL: https://www.zilog.com/docs/z80/um0080.pdf.

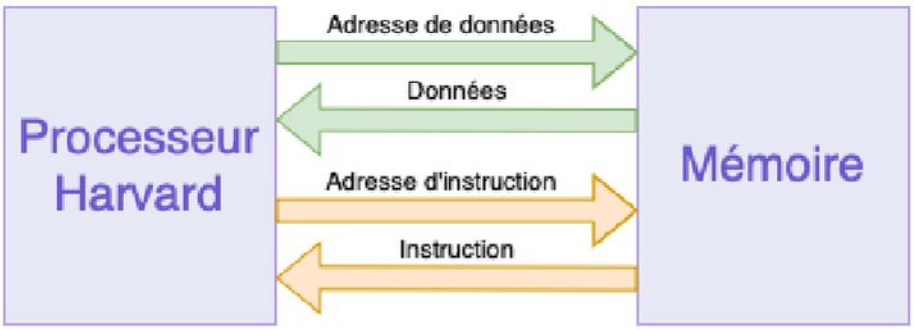


Figure 6: A processor with a Harvard-type architecture and its data buses

3.5 The use of an FPGA

Nowadays, a processor is implemented using an integrated circuit. To manufacture a chip, a lot of money is required (more than 10 million francs[29]). Limited by our budget, we therefore decided not to manufacture one, but to implement it on a reprogrammable circuit. Indeed, we used an FPGA (Field Reprogrammable Gate Array), an integrated circuit whose logic gates can be reconfigured to realize any logic circuit, for example a processor. However, more transistors per logic gate are needed so that the gate can be reprogrammable, which is why FPGAs are much slower than a normal integrated circuit, on which the transistors cannot be reconfigured.

The use of an FPGA is very important in order to be able to test the processor without having to remake it every time, and to make a prototype of an integrated circuit. When the prototype circuit is ready and validated, it is then manufactured in silicon on a chip. This method is used by almost all companies that manufacture chips, to avoid the loss of money and time in the event of an error. A major example of an error would be the bug in Intel's Pentium processor, which got divisions wrong due to an error in a lookup table.[30]

[29]Brian Bailey - semiengineering.com. *What Will That Chip Cost?* Page consultée le 10 Juillet 2024. URL: https://semiengineering.com/what-will-that-chip-cost/.

[30]Vaughan Pratt. *Anatomy of the Pentium Bug.* Page consultée le 10 Juillet 2024. URL: http://boole.stanford.edu/pub/anapent.pdf.

4 Operation

4.1 Glossary

A bit is a digit that can be either 0 or 1. A byte, composed of 8 bits, is therefore an eight-bit number. In our case, one word of our processor is 64 bits, or 8 bytes.

The hexadecimal format is the base-16 representation of a number. For example, the number 11 in base 10 is represented by B in base 16. Hexadecimal digits from 10 onward are represented by the letters A through F. This format translates well into binary since it is enough to convert each hexadecimal digit into 4 bits.

A data bus is a set of wires that allows data to be transferred between the different components of a computer. For example, 4 bytes of data could travel through a data bus 4 bytes wide. A bus connected to two different units can transfer, for example, an address or data.

4.2 Preliminary concepts

First note that every processor has a program counter, which counts from 0 and advances by one except during a "jump" to another address. Like any processor, at startup our processor receives a *RESET* signal to initialize the program counter to 0, which we have called *PC* for *program counter*. To implement jumps, we provided our processor with a second program counter that we have called *PC_NEW*. During normal operation, outside a jump, the counter *PC_NEW* is set to the value of the first program counter plus one. During a jump, the counter *PC_NEW* is set to the address of the desired jump. Finally, at each clock edge, the program counter is set to the value of *PC_NEW*.

Let us first talk about registers. A register, in general, is a very small storage space in which one can save a number, or a little bit of data. Our processor is equipped with 16 registers, each of which has a 64-bit storage space so that the lengths of a register and of an instruction coincide.

An instruction is a string of bits that specifies an operation to be performed. For example, this operation can consist of adding two registers of the processor together, comparing two numbers in order to set a flag to 1. This operation can also tell the processor to continue its execution of instructions starting from a given address (this is called a *jump*). This instruction can also signal the processor to transfer data between the different memories, such as the register memory and the RAM. This operation can also mean to do nothing at all.

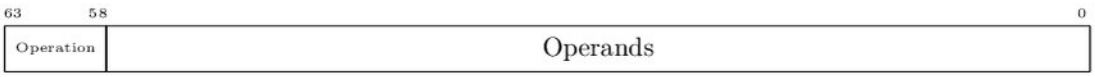


Figure 7: *Bit field of an instruction in our processor*

Note that, in our case, we decided to represent binary data in little-endian form. This means that, as shown in the previous diagram (Figure 7), the most significant bits are on the "left" of the expression.

Indeed, the last 6 bits of the instruction define the operation to be performed (Figure 7). Then, parameters such as the register number, such as a few addresses, or such as data, will be represented on the remaining bits. We chose to make the last 6 bits of the instruction the operation number because of the number of our instructions: during development, by optimizing the instruction set, we arrived at 38 instructions. Since $6 = \lceil log_2\ 38 \rceil$, we decided to use 6 bits for the operation number, as stated in the previous part. The instruction set appears in the appendix (Part 12.5). Since our processor does not stall under any instruction, the processor therefore executes one instruction per clock cycle.

4.3 Implementations

Our simulation and synthesis software, Vivado, computed a maximum clock speed of 40MHz for our implementation. We also know that our screen requires a clock speed ranging from 40MHz to 60MHz (with a recommended speed of 50MHz). This is why the clock speed of our processor is 40MHz.

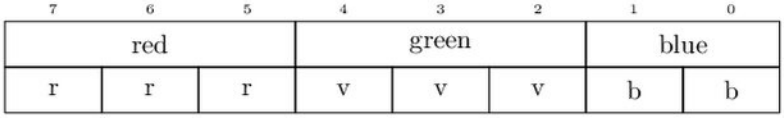


Figure 8: *Representation of a color in our VRAM*

The *RAM*, also called *main memory*, is the memory in which the data and the program will be stored. The *VRAM* is the graphics memory, used by modern graphics cards to store information related to graphics computation.[31] We used it to store the image sent to the screen, that is, the color of each pixel on the screen. Since we are limited to 225KB of memory blocks on our chip, we resorted to 128KB for the RAM and 64KB for the VRAM, to round to powers of 2. Since we only have 64KB of VRAM, we decided to represent each color in a single byte (3 bits for red, 3 bits for green, and 2 bits for blue, Figure 8). This makes it easier for us to modify pixels on the screen, because we implemented an instruction used to modify a byte in memory. We decided to limit the addressable part of the screen to dimensions of 512 x 480 physical pixels, which is easily representable in powers of two. Since the most convenient memory blocks (RAM blocks) are 14 bits wide, they can store 16KB of data, and since we did not have enough, we decided to use 4 of them, which amounts to saying that we decided to use 64KB of VRAM. But, since $512 \cdot 480$ is around 256KB, we could not afford to exceed the limit of RAM installed directly on the chip (225 KB). This is why we resorted to pixels that are twice as large. The logical size of the screen is therefore 256 x 240. To implement the 128KB of RAM, we used 8 separate RAM units for each byte, to simplify byte-wise memory storage (for the *mov*, *mem*, and *memi* instructions, as stated in Part 12.5).

4.4 Details of processor operation

At each clock edge, the processor reads the instruction at the address of the program counter. Then, this instruction enters the instruction decoder of the processor. Next, depending on the instruction, this decoder sends control signals to the various units of the processor, such as, for example, the arithmetic logic unit, the address unit, and the memory load/store unit. It also controls the flags that the instruction should or should not modify, by sending signals into the register and flag memory. In our case, the internal addresses of the registers requested by the instruction will be stored in the buses named *ADDRX* and *ADDRY* for the registers to be read, and *ADDRD* for the destination register. In addition, the instruction decoder sends signals outside the processor to control the external memory through the memory control unit, which is located outside the processor. This memory control unit switches between the different memories, such as the *RAM* or the *VRAM*, which contains the image that is displayed on the screen in our case. The instruction decoder also sets the address of the source and destination registers for computations, and also sets a number sometimes present in the instruction, called an immediate, which allows us, for example, to add numbers to registers in a single instruction, instead of putting a number into another register and then adding them together.

If the received instruction is an arithmetic instruction or the modification of a register's contents (this refers to the ldi instruction), the ALU is the one that will compute the result, then give the result to the register memory. To do this, the instruction decoder will send a *NOP* (do nothing) signal to all units except the ALU. In our case, to represent *NOP* in the processor's signals, we chose 0 as the signal. When the received instruction requires a jump to be made, it is the address unit that will do the work. As stated previously, the configuration of the signals will be the same (only this unit will see a signal that forces it to do something). When the instruction requires a data transfer, it is the memory load/store unit that will handle it. Finally, when the instruction requires the execution of an operation that will alter a register's memory, the internal flag named *flag_write_enable* will be activated, to tell the register memory to modify the register at address *ADDRD* using the result it receives.

4.5 Parts of the processor

Below (Figure 9), we have represented the connections of the various components of the processor. We will now describe each of the modules in detail. The signals shown in orange are address signals. The signals colored in blue are control signals, while the signals colored in pink are one-bit control signals. The signals shown in green are raw data, and the signals in black are global control signals, i.e., only the clock (*clk*) and the reset (*rst*).

[31] Alan Bradley, PC MAG. *What Is VRAM? Why Games Are Using it More, and How to Optimize Your PC*. Page consultée le 3 Juin 2024. URL: https://www.pcmag.com/how-to/what-is-vram-why-games-are-using-it-more-and-how-to-optimize-your-pc.

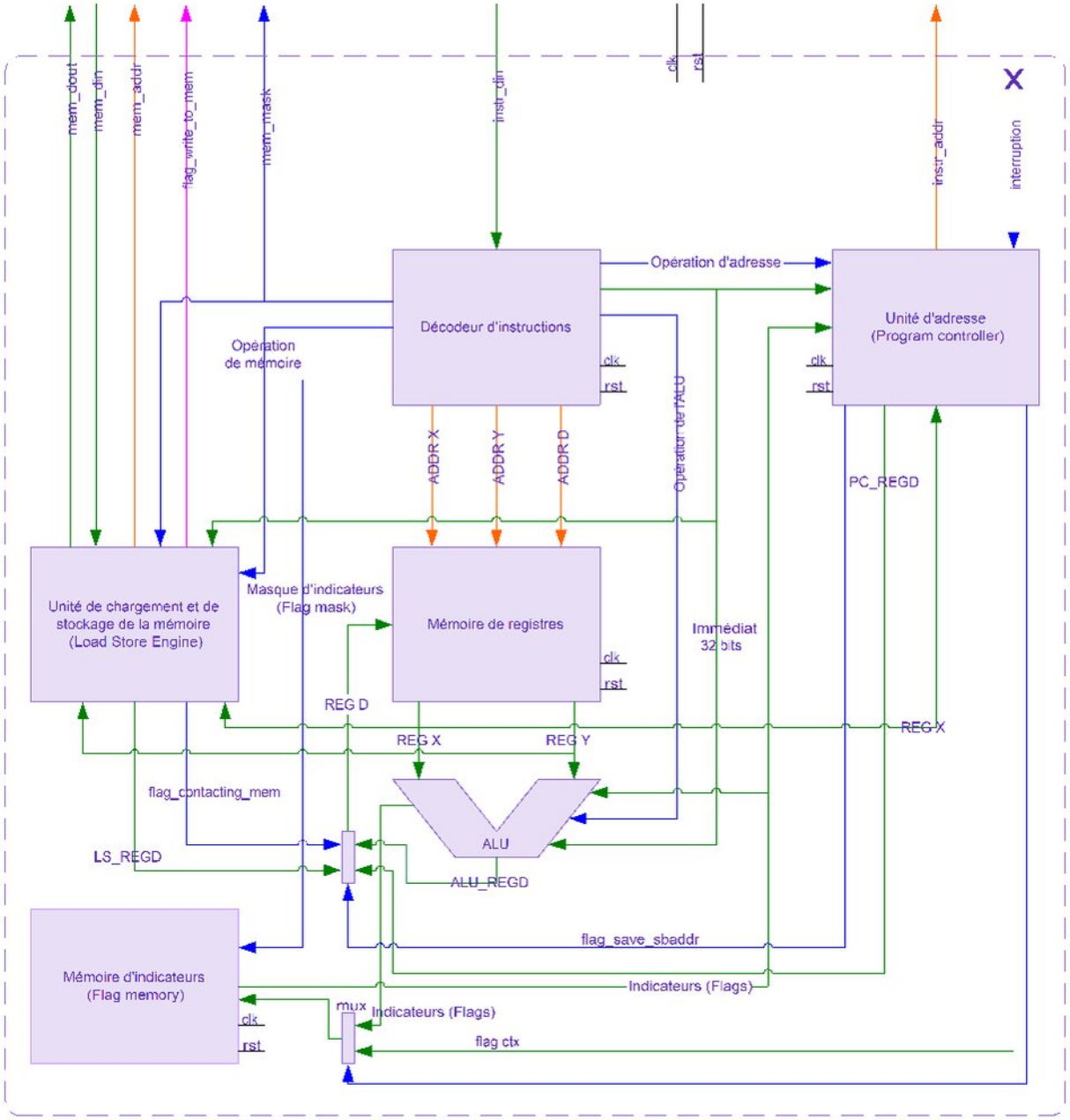


Figure 9: *Flowchart of the inside of our processor*

As explained above, the instruction decoder sends control signals to the various units of the processor depending on the instruction received (in blue).

The register memory receives the source and destination register addresses, where the address of register Rs_1 is named *ADDR X* on the diagram and the address of register Rs_2 is represented as *ADDR Y*. The address *Rd* is named *ADDR D* on the diagram. What is contained at addresses *ADDR X* and *ADDR Y*, i.e., *REG X* and *REG Y*, enters the ALU so that computations can be performed on them (for example, to add them together). *REG X* and *REG Y* also enter the unit that handles external memory, to then be sent into memory after modification, depending on the control operation (signal named "Memory operation" on the diagram). *REG X* is also sent to the address unit in order to perform jumps. The memory control unit decides, based on the received control signal, whether to save *LS_REGD* or *ALU REGD* by modifying *flag_contacting_mem*. The instruction decoder decides whether to save the *REG D* result or not.

A 32-bit immediate enters the ALU and the address unit so that various operations can be performed using a number directly, as mentioned in the description of the instruction set (see Appendices, chapter 12.4).

First, to store the flags, the instruction decoder decides whether it is necessary to do so for each of the flags or not (regulated by the *Flag mask* signal). Then, the address unit selects the result to be saved (whether its own or that of the ALU). Generally, the address unit only requests the saving of its version

of the flags (named *flag ctx*) when the instruction decoder receives the instruction to restore the flags (after the end of a subroutine's execution, for example).

We also indicated the location of the interrupt signal, had we implemented it.

Every non-combinational unit (i.e., a unit that only generates control signals, such as the instruction decoder, for example) receives a clock and a reset signal in order to be able to store information.

Finally, here is the description of the various signals going in and out of the processor. *mem_dout* is the signal going out to the external memory containing the data that the processor wishes to write at address *mem_addr*. *mem_din* is the signal entering the processor that contains the data at address *mem_addr*. The signal *flag_write_to_mem* gives the green light to the memory to write *mem_dout* at address *mem_addr*. The signal *instr_din* contains the instruction at address *instr_addr*.

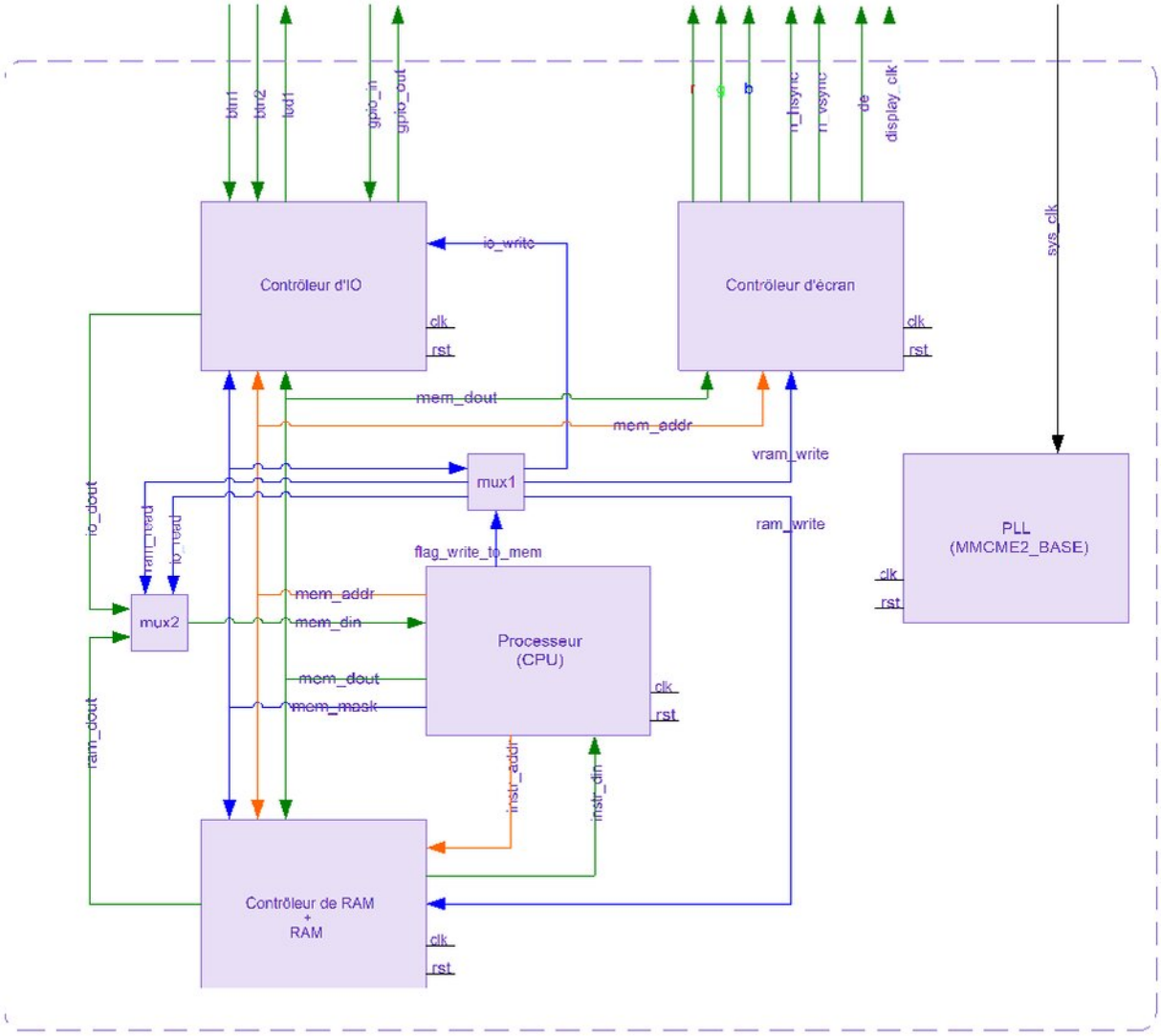


Figure 10: *Flowchart of how our processor is connected to the various peripherals*

The processor is connected to the RAM controller (as shown in Figure 10), to the IO controller, i.e., the one that handles the inputs and outputs of the system-on-chip (i.e., of the Cmod A7 that we use), and to the screen controller, which generates the signals necessary for the proper operation of the screen. To switch between the different units, we placed two multiplexers called *mux1* and *mux2* on the diagram. These are not necessarily Verilog modules in the system's source code, but are implemented directly in the `wrapper.v` file. To switch between the different units, we decided to use the address as a unit selector. If the 32ème bit is set to 1, then the IO controller is selected, that is, a store operation will result in the modification of the system's outputs. If the 64ème bit is set, then the screen controller will be selected. A store operation will therefore result in a write to the VRAM, and thus in the modification of pixels on the screen.

For simplicity, here are the computation formulas:

$$ram_write = (mem_write \wedge \overline{mem_addr_{32}}) \wedge (\overline{mem_addr_{64}})$$

$$io_write = (mem_write \wedge mem_addr_{32}) \wedge (\overline{mem_addr_{64}})$$

$$vram_write = mem_write \wedge mem_addr_{64}$$

These signals give the green light to the respective unit to save data. Note that, in these formulas, as in the code, *mem_write* corresponds to the *flag_write_to_mem* signal. Here is the formula for the signals that choose from which unit the data should be read (the VRAM, representing the pixels on the screen, is write-only):

$$ram_read = \overline{mem_write} \wedge \overline{mem_addr_{32}}$$

$$io_read = \overline{mem_write} \wedge mem_addr_{32}$$

We use a phase-locked loop (PLL) to generate the clock and the reset signal (*rst*). The signal *clk* is the system clock, and the signal *rst* is the processor's reset signal. The signal *rst* is set to 1 for a few clock cycles, then goes back to 0. This allows the registers and the counters to be reset, to put the processor back into a stable state.

In the same way as for the previous diagram, we will describe the incoming and outgoing signals of the system. *btn1* is the value of button 1 on the Cmod A7 board. When the user presses button 1, a value of 1 can be read from *btn1*. Same for *btn2*. When *led1* is set to 1, then LED 1 on the Cmod A7 board lights up; otherwise, it turns off. *gpio_in* is composed of 8 bits and corresponds to pins 33 to 40 inclusive (for example, the first bit designates pin number 33) of the Cmod A7. *gpio_out* is also composed of 8 bits and corresponds to pins 41 to 46 inclusive of the Cmod A7. The last two bits of *gpio_out* are not used. Pins 47 and 48 are the inverse of each other and are PWM outputs (Pulse Width Modulation) with a resolution of 8 bits. This means that we can send values from 0 to 255 to control the width of the PWM pulse. Since the clock speed of our processor is 40MHz, the PWM frequency is 40MHz/256 = 156.25kHz.

The processor will be able to read *gpio_in* and modify *gpio_out* through the IO controller. Here is the mapping table between external peripherals and what the processor must put in memory, which will apply in the same way to *mem_din* and *mem_dout*:

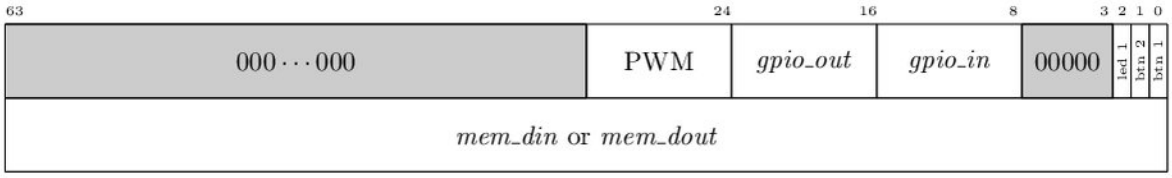


Figure 11: *Bit field between mem_din, mem_dout and the external peripherals*

Then, *r*, *g*, and *b* control the color of a pixel on the screen. To authorize this modification, the signal *de* is set to 1. To send an image to the screen, *hsync* is set to 1 while transmitting data line by line. Before and after each data send for a line, there is a short pause during which *hsync* is set to 0. During the transmission of lines, *vsync* is set to 1. Also, before and after the transmission of all lines, *vsync* is set to 0. Finally, *display_clk* is simply connected to *clk*, because, as mentioned at the beginning of subsection 4.3, the speed of our clock matches that of our screen.

Finally, *sys_clk* is used to control the phase-locked loop (PLL) that will generate the main clock (*clk*).

4.6 Prototype



Figure 12: *Cmod A7, the module on which we built our processor*

To implement everything, we decided to use a printed circuit board made by *Digilent*,[32] which houses an FPGA from *Xilinx*.[33] To send the logic-gate configuration onto our chip, the FPGA, we used *Vivado*,[34] a software by *Xilinx, AMD*. Indeed, we therefore chose the *Digilent Cmod A7*[35] (Figure 12), with the *Xilinx ARTIX-7 35T cpg236-1* chip[36] because this printed circuit board was not very expensive (around 90 CHF) (Figure 12).

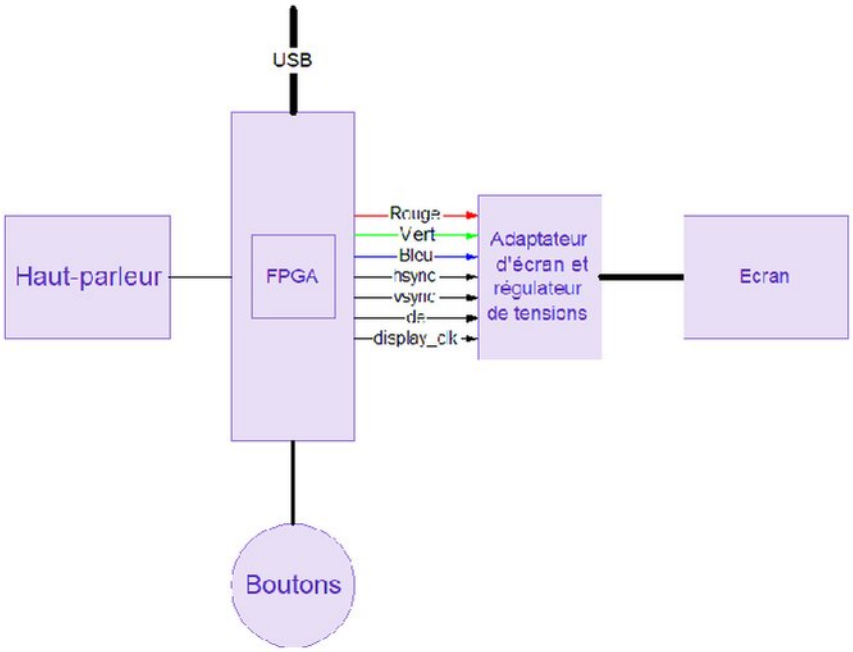


Figure 13: *Flowchart of our prototype*

To connect it to the screen, we used a screen adapter with a voltage converter (Figure 13) that we connected to the *Digilent Cmod A7* (Figure 14). For sound, we connected a speaker to the *Digilent Cmod A7*. For power, we used a USB cable. To interact with our system, we used 4 push-buttons with 4 built-in LEDs. The screen,[37] the voltage converter,[38] and the buttons[39] were purchased from *Adafruit*. The speaker[40] was purchased from *Mouser*.

In our implementation of Tetris, the buttons are connected to the first 4 "input" GPIO pins (pins 33, 34, 35, and 36), while the LEDs are connected to the first 4 "output" GPIO pins (which correspond to pins 41, 42, 43, and 44). The two speaker pins are connected to the last two "output" GPIO pins (pins 47 and 48). The *h_sync* and *v_sync* signals are connected to pins 1 and 2 respectively. The *de* and *clk* signals come out of pins 3 and 4 respectively. The "red" bus comes out of pins 5 to 12; the "green" bus comes out of pins 13, 14, and pins 17-22 (pins 15 and 16 were skipped since they are analog inputs). The "blue" bus comes out of pins 23, 26, and pins 27-32 (pins 24 and 25 were skipped since they are power pins).

[32] Digilent. *Digilent – Start Smart, Build Brilliant.* Page consultée le 1er Juin 2024. URL: https://digilent.com/.

[33] Advanced Micro Devices, Inc. *Adaptive SoCs and FPGAs.* Page consultée le 31 Mai 2024. URL: https://www.amd.com/en/products/adaptive-socs-and-fpgas.html.

[34] Advanced Micro Devices, Inc. *Vivado Overview.* Page consultée le 31 Mai 2024. URL: https://www.xilinx.com/products/design-tools/vivado.html.

[35] Digilent. *Cmod A7-35T: Breadboardable Artix-7 FPGA Module.* Page consultée le 31 Mai 2024. URL: https://digilent.com/shop/cmod-a7-35t-breadboardable-artix-7-fpga-module/.

[36] Advanced Micro Devices, Inc. *AMD Artix™ 7 FPGAs.* Page consultée le 31 Mai 2024. URL: https://www.amd.com/en/products/adaptive-socs-and-fpgas/fpga/artix-7.html.

[37] Adafruit. *7.0" 40-pin TFT Display - 800x480 without Touchscreen.* Page consultée le 15 Juillet 2024. URL: https://adafru.it/2353.

[38] Adafruit. *40-pin TFT Friend - FPC Breakout with LED Backlight Driver.* Page consultée le 15 Juillet 2024. URL: https://adafru.it/1932.

[39] Adafruit. *Mini LED Arcade Button - 24mm Translucent Clear.* Page consultée le 15 Juillet 2024. URL: https://adafru.it/3429.

[40] Mouser Electronics, Inc. *GF0876 CUI Devices.* Page consultée le 15 Juillet 2024. URL: https://www.mouser.ch/ProductDetail/CUI-Devices/GF0876?qs=WyjlAZoYn5229XpWUmpA%252Bg%3D%3D.

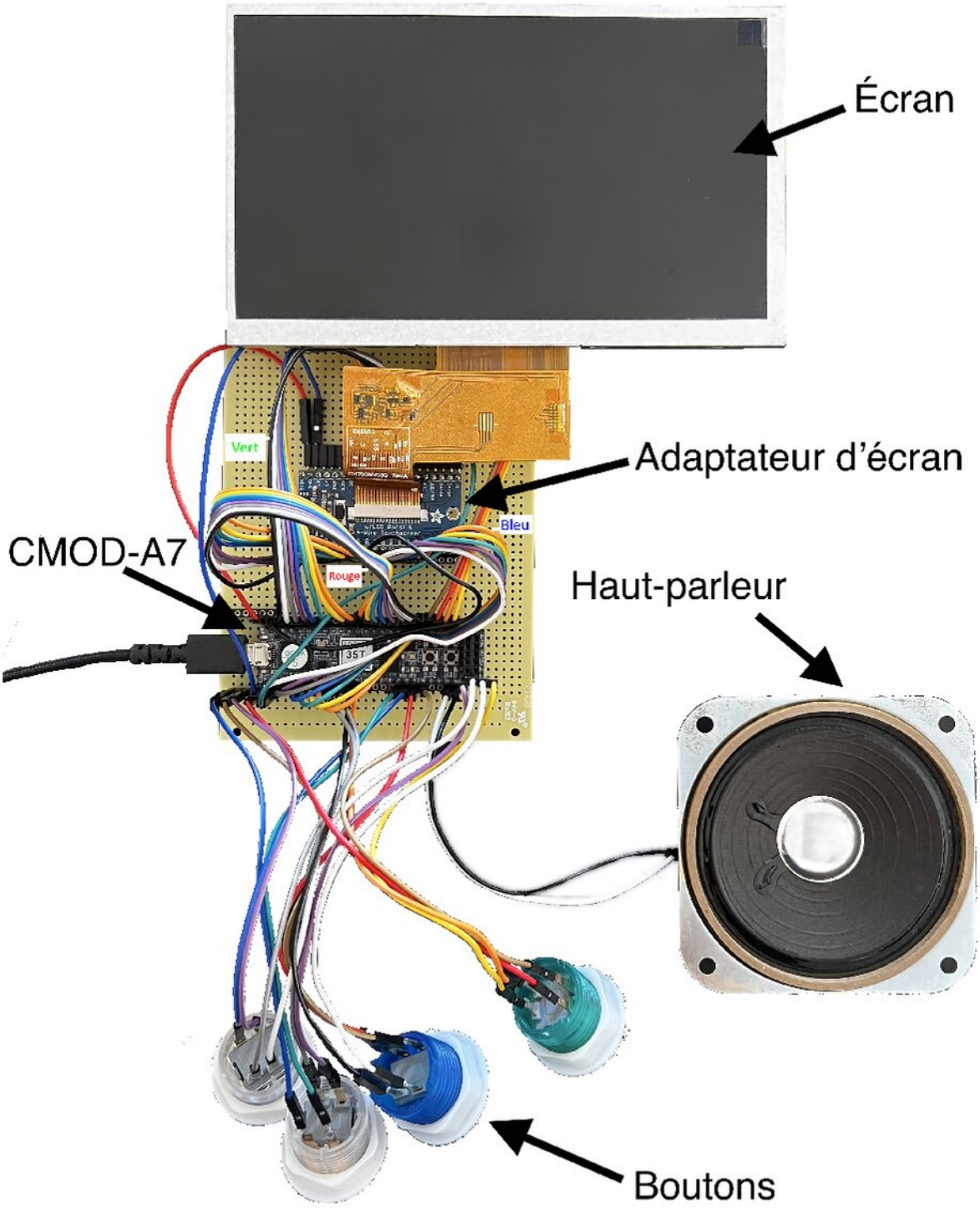


Figure 14: *Our prototype, which is a complete computer*

4.7 A few details

To debug our processor, we implemented a *test bench*, that is, a program that simulates the various inputs and outputs required by the processor in order to function. Then, in *Vivado*, to see the processor's internal signals, we simulated it (Figure 15).

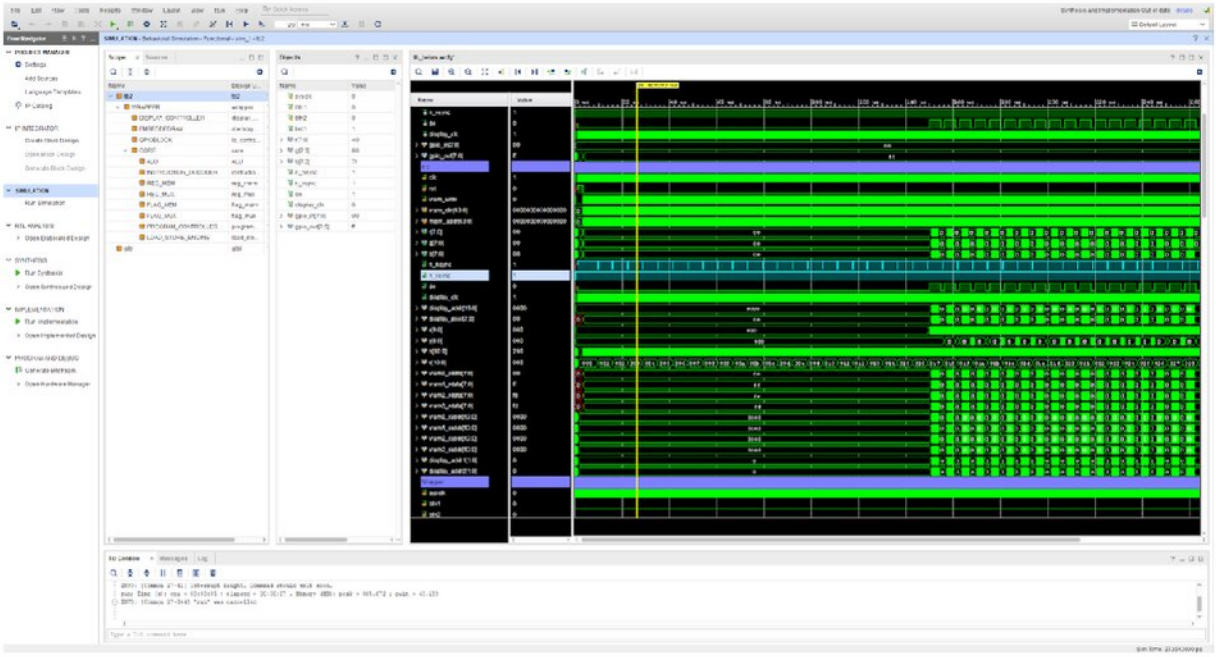


Figure 15: *A screenshot of the Vivado software after simulation. The n_hsync and n_vsync signals, used to synchronize the sending of data to the screen, are shown in light blue. The software allowed us to debug their timing.*

5 Programming the Assembler Compiler

5.1 Introduction to assembly

Most programmers write code in high-level languages, that is, languages built to be easily understood by humans (such as Python or C, for example). What the machine understands is generally a low-level language, as in our case. Our processor accepts binary instructions (also called *machine code*) and not high-level language. This is why, to make the task of writing programs for our processor easier, we designed a compiler, that is, a program that will translate a program written in assembly, which is a very low-level language[41], into machine code.

Our compiler translates assembly code (Figure 16) into machine code, i.e., into binary code. Note that each assembly instruction corresponds directly to one binary instruction in machine code.

The assembly format is as follows:

xxx: defines a label named *xxx*

;xxx defines a comment and is ignored by the compiler

**$**$r_x$ is register number x

x'hy is the number y in hexadecimal format, encoded in x bits

x'dy is the number y in decimal format, encoded in x bits

x'by is the number y in binary format, encoded in x bits

#define a b defines a as b.

@a is replaced by the definition of a. If the command above were detected, then "@a" would be replaced by "b".

dw uses the format of a normal instruction, except that it is one for the compiler and not for the processor. This command accepts 8 bytes of data and is used to define them. We chose to call it "*dw*" for *define word*.

ds works like *dw*, except that it only accepts an 8-byte character string. We chose to call it "*ds*" for *define string*.

[41] Assembly, specific to each processor, is a language that is very close to machine code. It is nevertheless more understandable to programmers than the latter.

```
main:
    ldi $r0 32'b0 1'b0
    ldi $r0 32'b0 1'b1
    ldi $r1 32'b0 1'b0
    ldi $r1 32'b0 1'b1
    ldi $r2 32'd512 1'b0
    ldi $r2 32'b0 1'b1
loop:
    addi $r0 $r0 32'd1 4'b0
    cpi $r0 32'h7FFFF
    bifr 32'hA0000003 ;jump across jmpi
    nop
    jmpi loop
    nop
blink:
    nop
    ldi $r0 32'b0 1'b0
    ldi $r0 32'b0 1'b1
    xori $r1 $r1 32'hFFFFFFFF 1'b0 ; toggle led
    mem $r2 $r1 32'd0 8'b11111111 ; upload new led status to pin
    jmpi loop
    nop
```

Figure 16: *Example of assembly code. By executing this code, the processor blinks the LED of the external button connected to the first input and output pins of the Cmod A7 board.*

5.2 How the compiler works

To achieve this goal, we coded a Python script capable of compiling code written in assembly into machine code. First, comments and whitespace are completely removed from the program. Next, a list is initialized with each instruction, line by line. Then, the Python script does a first pass, searching for a macro (an abbreviation defined by the user with the *#define* keyword) and labels (i.e., to create aliases for addresses, for example to implement a function). Upon finding these, it stores the address of each of them in a dictionary. Then, it does a second pass to detect the instructions and their addresses. In doing so, it recomputes the addresses of each instruction, because a label does not count as an address (a label is only an indication to the compiler). For each instruction found, it converts it into binary format based on its arguments. Arguments that designate labels are replaced by the addresses computed for them. All these new instructions in binary format are stored in a list and, finally, written to 8 files in hexadecimal format, to later be loaded into the 8 RAM modules at system power-up (we implemented 8 RAM units, as described in Part 4.3).

6 Programming Tetris

To program Tetris, we first needed to invent an algorithm. This is why we first wrote it in the C programming language, because it is easier to debug than assembly (we can, for example, print variables to the console or watch the values of variables live). C is a language that compiles to assembly to then be executed by the processor.[42] In this language, we implemented the algorithm for the game Tetris.

As a preliminary, let us discuss what Tetris is. In the 1980s in the Soviet Union, Aleksey Pajitnov, a researcher in the field of artificial intelligence and in the field of computer speech recognition, developed the legendary game Tetris. It is a game made up of tetrominoes (game pieces consisting of four squares) and a rectangle measuring 10x20 squares. Each piece descends little by little from the top of the rectangle, and during this descent, the player can control the piece by rotating it or by moving it left or right. The piece stops when it stacks on top of another or when it reaches the bottom of the rectangle. The player can speed up the pieces, but can neither slow down nor stop them. So the goal of the game is to stack the tetrominoes so well that they form horizontal lines. When a line is filled, it disappears and everything that was above the line drops by one square. In doing so, the game awards points to the player.[43]

6.1 Programming in C

In order to later transcribe the code into assembly, we wrote the code as simply as possible. We therefore decided to use a list of 200 values ($10 \cdot 20 = 200$) to represent the rectangle that houses the tetrominoes in the game. In order to rotate the tetrominoes, the decision was made to include a pointer to the next rotation state of the tetromino. Note that each tetromino is composed of 4 squares, and that it has a unique color. Also, having a geometric origin for each tetromino allowed us to move it more easily (it was enough to increment the value x or y). With this origin, we used offsets for each square of the tetromino. So we used the following data structure: `(next_rotation*, sx1, sy1, sx2, sy2, sx3, sy3, color)`. With this data structure, to rotate the tetromino, it is enough to replace the pointer of the current tetromino with the pointer named *next_rotation*.

In Tetris, there are 7 possible tetrominoes (Figure 17), with 4 rotations at most. We decided to omit similar states so that the program would take up less space (after rotation, the piece could be the same as one encountered before). In the adjacent figure (Figure 17), we represented the geometric origin of each tetromino in a clearly visible color.

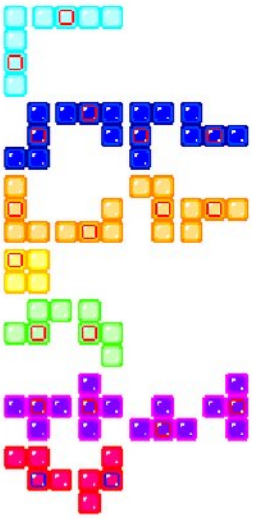


Figure 17: *The different types of tetrominoes: I, O, T, J, L, S, Z (from top to bottom)*

To be able to open a window and draw on it, we used the SDL library of C,[44] which we will not transcribe into assembly.

Each time the delay elapses or a button is pressed, the `step` function is executed. Within this function lies our main Tetris algorithm. When it is called, it removes the controllable tetromino from the screen. Then, it checks whether a button was pressed. If it was the button to rotate the piece, it checks whether the piece can undergo a rotation (by checking whether a square of the new state of the piece would touch a border or another piece). If so, it changes the piece's pointer to its next rotation state. After doing this, it puts the piece back on the screen (whether it changed state or not).

If the player wanted to move the piece to the left, it checks in the same way whether the piece would touch something. If not, it decrements the tx coordinate of the tetromino. Then it puts it back on the screen (whether it underwent a change or not). Same for the case to the right (except by incrementing tx).

If the player wished to drop the tetromino, it checks in a loop, after incrementing the ty coordinate, whether it would touch something. If so, then it puts the piece back on the screen (this time, since the coordinates will have changed, permanently), then sets the tx and ty coordinates to $(tx, ty) = (5, 0)$. Finally, it randomly chooses a new tetromino, then places its pointer in memory in the *current* variable, which designates the current tetromino.

If the player had not pressed anything, the `step` function examines whether the piece could be moved down by one square. If it can, it increments the ty coordinate while drawing the tetromino on the screen.

[42] codecademy.com. *Compiling.* Page consultée le 23 Juin 2024. URL: https://www.codecademy.com/resources/docs/c/compiling.

[43] Semenov Sergei - histrf.ru. *How a Soviet programmer made the worlds most popular game.* Page consultée le 23 Juin 2024. *Titre traduit.* URL: https://histrf.ru/read/articles/skaz-pro-to-kak-sovetskiy-programmist-samuyu-populyarnuyu-igru-v-mire-izobrel.

[44] libsdl-org. *Simple DirectMedia Layer (SDL) Version 3.0.* Page consultée le 23 Juin 2024. URL: https://github.com/libsdl-org/SDL.

If this is not the case, it places it on the screen indefinitely (for the same reason as above), then re-centers the tx and ty coordinates, and chooses a new tetromino (same algorithm as for "dropping the piece").

Finally, if a line has been completed, the function shifts the entire rectangle down by one square. If we had implemented a scoring system in C, we would have the possibility at that point of awarding points to the player.

This program in C contained only the basic algorithm of Tetris. This is why, in the image below (Figure 18), there was nothing other than the 10x20 rectangle.

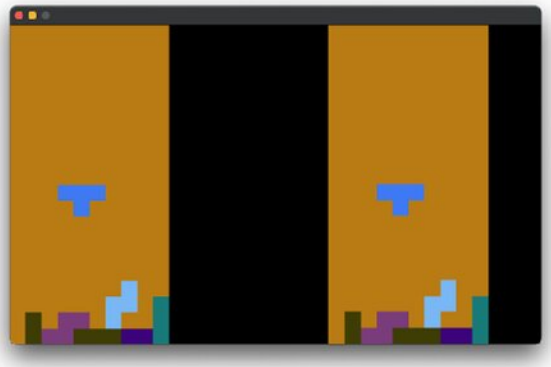


Figure 18: *A screenshot of Tetris programmed in C.*

6.2 Transcription of the algorithm into assembly

Some parts of the algorithm written in assembly differ from the one written in C. For example, to check whether there is a collision, the one in C checks for each square of the tetromino whether it touches another square of the rectangle. In assembly, we decided to copy the rectangle into another part of the memory, called *rectangle_copy*. Next, we removed the squares of the tetromino in *rectangle_copy*. Finally, we checked whether *rectangle_copy* had changed by being modified by the tetromino. If so, then there is a collision.

The Tetris algorithm written in assembly is shown below (Figures 19, 20, and 21).

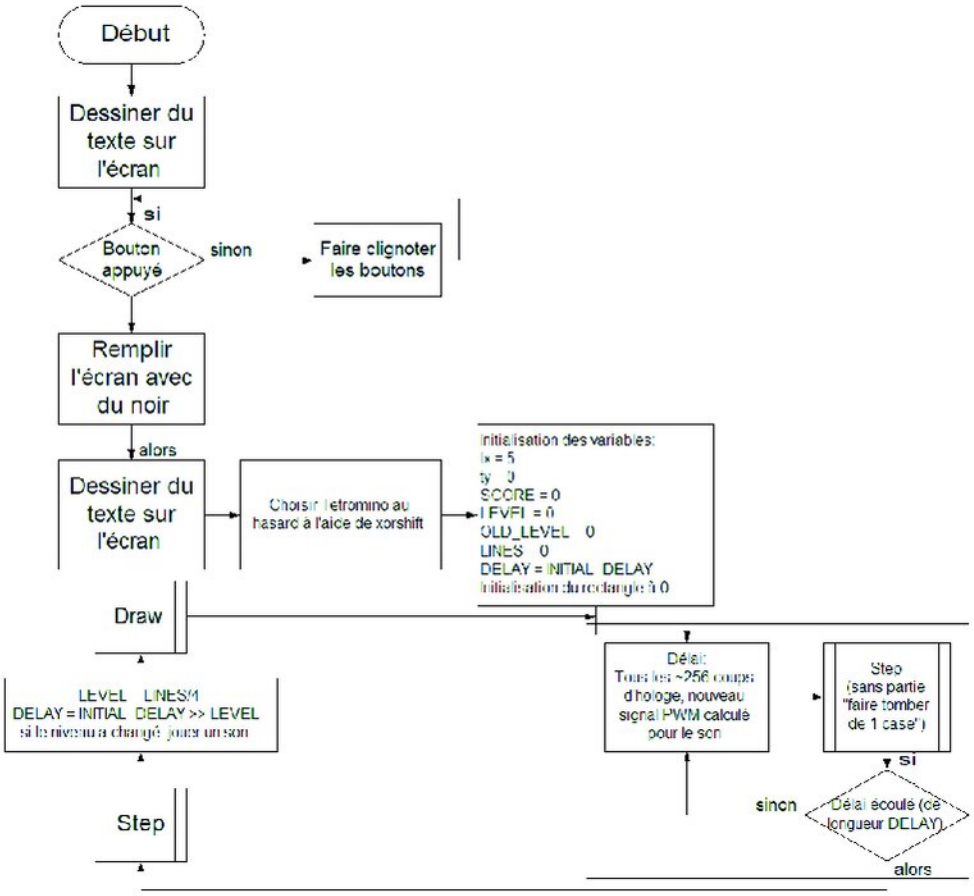


Figure 19: *The Tetris algorithm written in assembly.*

The variable *SCORE* is incremented by 25 each time a line is completed. When the player presses "Drop", then the variable *SCORE* is incremented by 1 for each square that the tetromino passes while falling. *LINES* is incremented by 1 each time a line is completed. The variable *DELAY* is divided by 2 each time *LINES* is a multiple of 4. We will want to convert the score represented in two's complement into text in order to display it on the screen without using any complicated operation. To do this, we will manipulate the bytes so that they contain a value from 0 to 9. This form of number storage is called "binary-coded decimal" (abbreviated BCD). Note that this correction method only works when adding two numbers in BCD format. This approach was chosen over another because it requires no multiplication or division that could slow down the program.

The correction is as follows: we first convert the number into BCD format (for example, 0000_2 would be converted to "0", 0101_2 would be converted to "5"). Then, when a hexadecimal symbol (the hexadecimal format represents numbers in base 16; for example, $01010010_2 = 52_{16}$ and $01110011_2 = 73_{16}$) is greater than 9, we add 6 to it to obtain the corresponding digit in base 10, and so on for the more significant symbols. The total score is saved in hexadecimal format so that it can then be displayed directly on the screen. Each incrementation of the score therefore requires a correction of this type, as illustrated in the following examples.

By way of illustration, if the initial score was 19_{16} and we wish to add 1, we will have $19_{16}+1_{16} = 1A_{16}$. However, since the first symbol is greater than 9, we add 6 to correct the sum. The calculation therefore becomes $1A_{16}+6_{16} = 20_{16}$, which is indeed the sum of 19 and 1 in the decimal system ($19_{10}+1_{10} = 20_{10}$).

Another example would be 99_{16}, the current score, to which we would like to add 1_{16}. We will have $99_{16} + 1_{16} = 9A_{16}$. Since the first symbol is greater than 9, we add 6 as in the previous example. The calculation becomes $9A_{16} + 6_{16} = A0_{16}$. Then, we perform the same operation for the second symbol since it is greater than 9. We will therefore add the correction of 60_{16} to modify the second symbol. We have $A0_{16} + 60_{16} = 100_{16}$, which is the expected corrected sum for $99_{10} + 1_{10} = 100_{10}$. This cycle continues for each symbol greater than 9 in the number, up to the last symbol.

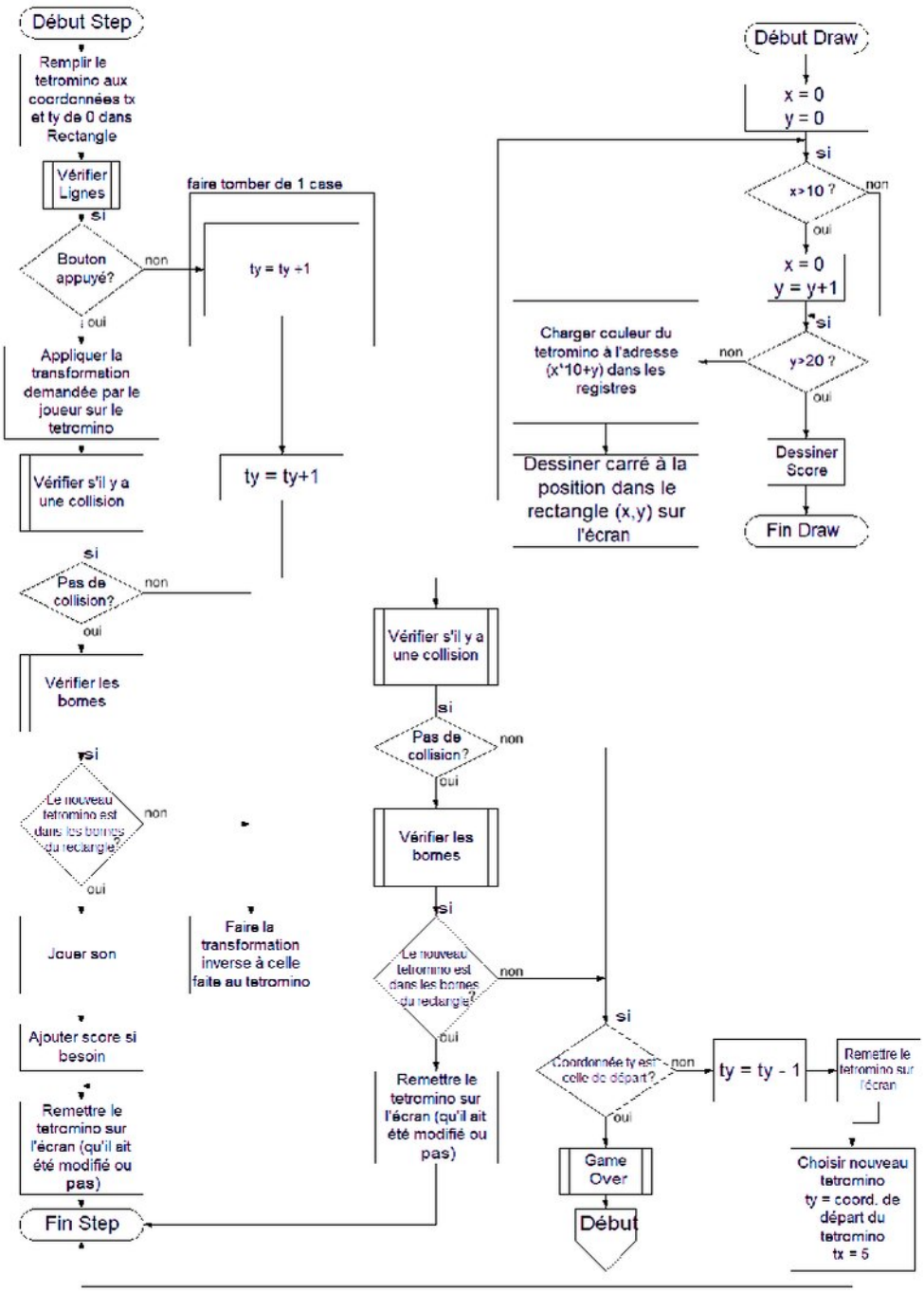


Figure 20: *Algorithm of the step and draw functions.*

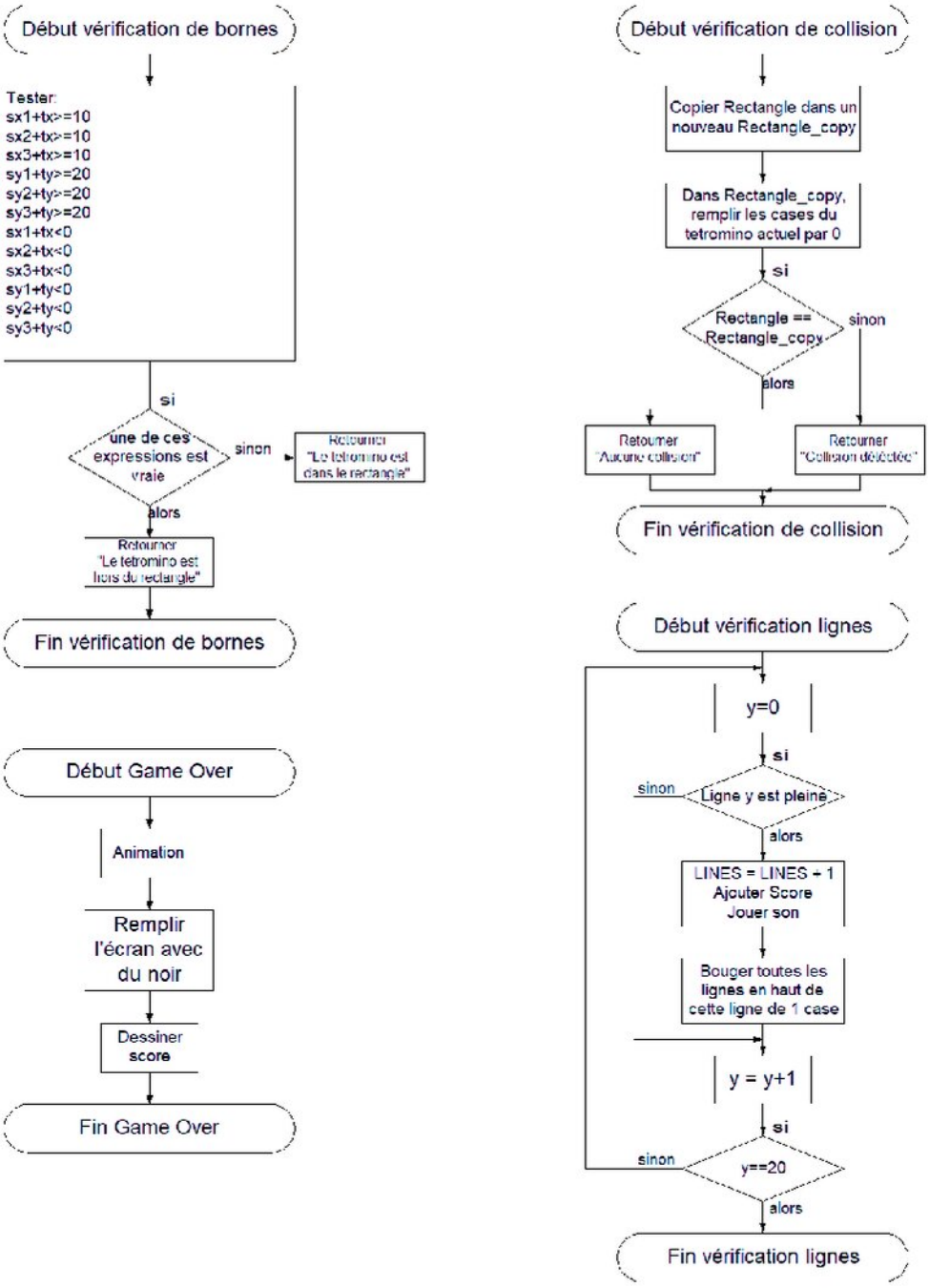


Figure 21: *Flowchart of Tetris written in assembly (continued).*

6.3 Memory organization



Figure 22: *Photo of the title screen.*



(a) *The IBM BIOS character font.*

(b) *Our character font for the digits.*

To write text on the screen (Figure 22), we imported an uppercase character font into memory. The dimension of each letter is 8x8 pixels (Figure 23a). To write the score on the screen, we created a 16x18 character font composed only of digits (Figure 23b).

7 Sound Implementation

7.1 Physical operation

To implement sound, we decided to equip the processor with a PWM output (explained in chapter 3.4). The value of each sample of the signal varies between 0 and 255, which is why we have 8-bit sound. The processor, at each clock edge, increments a counter that also goes from 0 to 255. If this counter is smaller than the value of the signal, the output is 1; otherwise, it is 0. This creates a square-wave signal, which is then sent to the speaker to produce a sound.

Since the clock oscillates at 40MHz, our maximum sampling frequency is $\frac{40'000'000}{256} = 156.25kHz$ (our system needs 256 clock edges to emit a square-wave signal, as stated previously). Indeed, our system can produce a sound with a maximum frequency of 78.125kHz, by the Nyquist-Shannon theorem.[45] This is not a problem, since the maximum frequency audible to humans is 20kHz.[46]

[45] Ayfer Ozgur. *Lectures 9-10: Sampling Theorem*. Page consultée le 12 Juillet 2024. URL: https://web.stanford.edu/class/engr76/lectures/lecture9-10.pdf.

[46] Jim Davis. *HEARING*. Page consultée le 12 Juillet 2024. URL: hhttps://iu.pressbooks.pub/humanphys/chapter/hearing/.

7.2 How a synthesized sound works



Figure 24: *A volume graph of a sound in Serum, a virtual synthesizer.*

A sound generally has an attack, a hold, and a release (Figure 24). Indeed, the attack is the time the sound takes to reach its maximum volume, and the hold is the time the sound stays at its maximum volume. Then, the sound decays toward its minimum volume. For simplicity, we implemented neither the attack nor the hold, which means that every sound starts playing instantly and stops progressively.

7.3 Computation of the sound

To implement the sound, we send a new value to the PWM outputs of the processor every `SOUND_FREQ` clock edges (as explained above). `SOUND_FREQ` is determined by the frequency of the sound we wish to play. If `SOUND_FREQ` is large, then the sound is low-pitched, and if it is small, then the sound is high-pitched. The method that sends the values to the PWM outputs is called `play_sound` and is called every 256 clock edges. This method, each time it is called, increments `SOUND_PROGRESS` (the playback time that increases over time) by 1, in order to know where it is in the sound. It also increments the variable `SOUND_PROGRESS2` each time it sends a value to the PWM outputs.

This method uses the variables `SOUND_PROGRESS`, `SOUND_PROGRESS2`, `SOUND_DECAY`, `SOUND_TYPE`, `SOUND_FREQ`, as well as `SOUND_TONAL_SHIFT`.

Let x be the next value to put in the PWM outputs. First, the sound is computed: if the sound is of the "noise" type, then x is generated randomly by the `xorshift` method (a randomly produced sound will sound like white noise, i.e., like the sound emitted by a radio when nothing is being broadcast). If the sound is of the "square" type, then $x = 255$.

Let $y =$ `SOUND_PROGRESS` $>>$ `SOUND_DECAY`. Then, x is shifted to the right by y ($x = x >> y$). Indeed, the larger `SOUND_DECAY` is, the less `SOUND_PROGRESS` will have an impact on x, and the longer the sound will last. Note that if `SOUND_PROGRESS` is a very large number, then no sound will be played.

Next, to create sound effects such as tonal shift, we added a variable `SOUND_TONAL_SHIFT`. This variable is added to `SOUND_FREQ` each time a new x is computed (this delay is managed by `SOUND_PROGRESS2`). This allows the frequency of the sound to be changed in real time.

Finally, x is sent to the PWM outputs.

If there is no sound to play and the sound type is "square", then $x = 0$ is sent to the PWM outputs, to allow a square wave to be generated.

7.4 Playing a sound

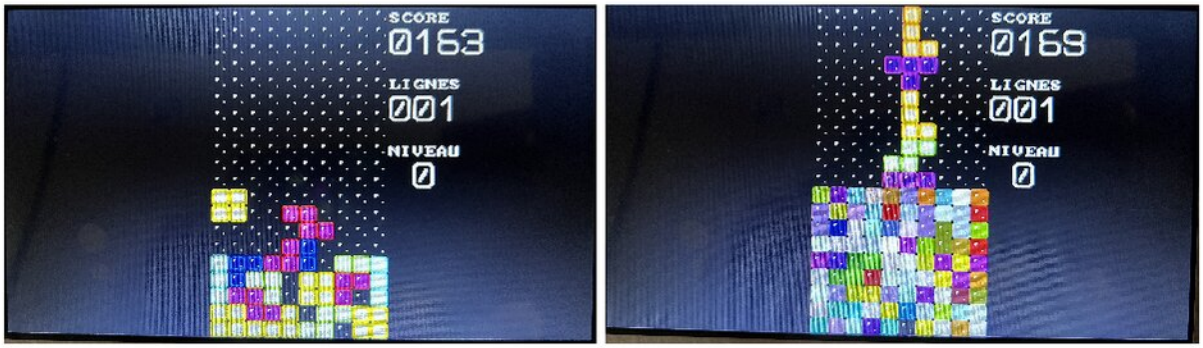
To play a sound, it is enough to reset the variables `SOUND_PROGRESS` and `SOUND_PROGRESS2` to 0, and to change the value of `SOUND_FREQ` to the frequency of the sound we want to play, `SOUND_DECAY` to the duration of the sound, and `SOUND_TYPE` to "noise" or "square", as well as `SOUND_TONAL_SHIFT` to the desired value. Then, since the `play_sound` method is already being called in a loop during the delays, the sound will be played.

8 Results obtained

8.1 Recap

In the photos below (Figures 25a, 25b, 27a, and 26a), we can see our computer running Tetris.

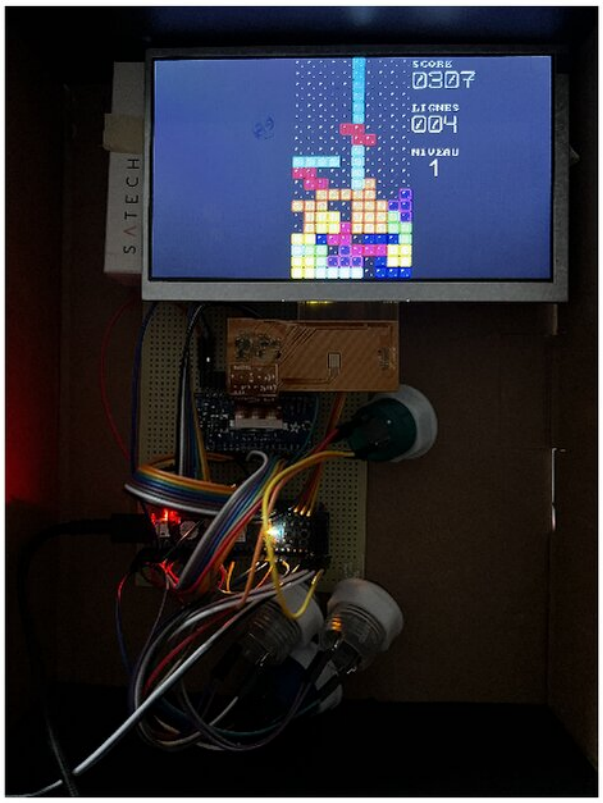
We also created a case that can be seen in Figure 26a.



(a) *The game Tetris running on our processor.* **(b)** *Game-over animation.*



(a) *Game over.*



(a) *Another photo of our processor running the game Tetris.*

8.2 Technical details

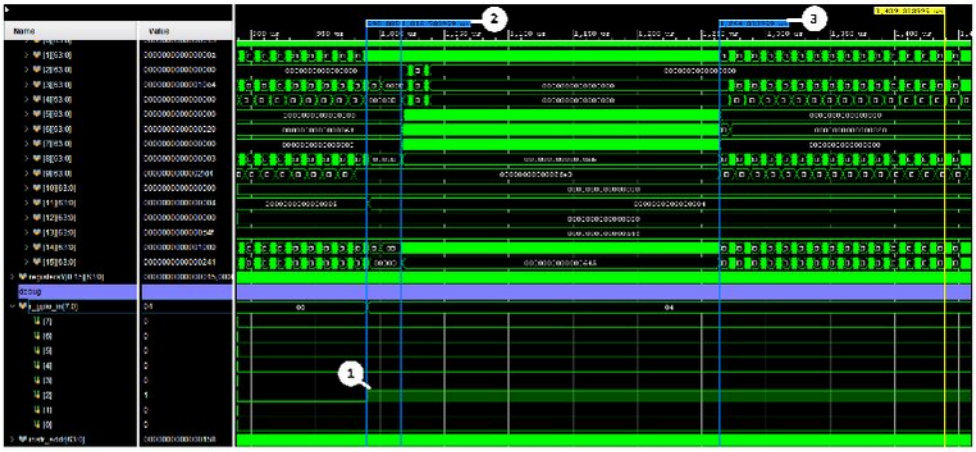


Figure 28: *Simulation of the processor running Tetris, reacting to a button press.*

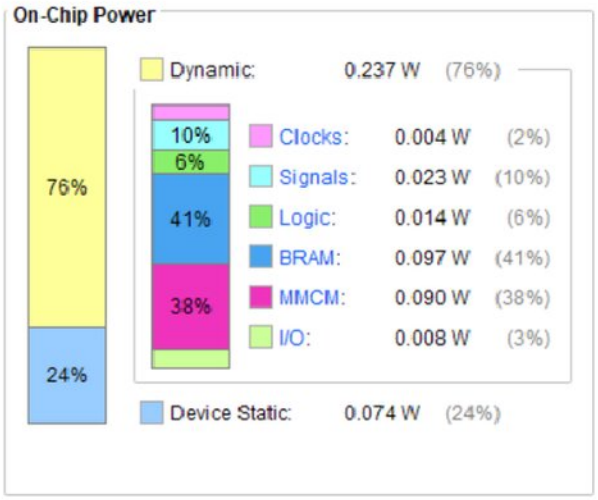


Figure 29: *Power consumption of the processor.*

To measure the time elapsed between a button press and the reaction of the processor running Tetris, we simulated it (Figure 28). We obtained a processor reaction time of $26\mu s$ ($1016 - 990 = 26\mu s$, latency between points 1 and 2) after a button press. After pressing the button, the processor computes the new position of the tetromino in $274\mu s$ ($1264 - 990 = 274\mu s$, latency between points 1 and 3), which is very fast.

Vivado gave us an estimate of the system's power consumption, which is $0.237 + 0.074 = 0.311W$ (Figure 29).

8.3 Comparison with other processors

Processor	Resolution [bits]	Clock speed	MIPS (millions of instructions per second)	MIPS/MHz	Multi-purpose registers
Our processor	64	$40MHz$	40	1	16 registers of 64 bits
STR71xF[47]	32	$50MHz$	45	0.9	31 registers of 32 bits
ESP32[48]	32	$240MHz$	[49]142	0.6	4 registers of 16 bits

Processor	Storing a 64-bit number [Clock cycles]	Adding two 64-bit numbers [Clock cycles]	General power consumption [W]	Maximum amount of RAM [bytes]	Architecture
Our processor	1	1	0.311	2^{63}	our own
STR71xF	> 2	> 2	0.2	$64KB$	`ARM7TDMI`[50]
ESP32[51]	4	> 4	$0.31 - 0.79$	$2^{32} = 4GB$	`RISC-V`

Table 1: *Comparison between our processor and a few current processors.*

To compute the number of MIPS/MHz (millions of instructions per second), we simply divided the number of instructions per second by the clock speed. Note that for the *ESP32* processor, we used a benchmark.[52] To compute the power consumption of the processors, we used the formula $P = U \cdot I$, so we multiplied the voltage and the current to measure the power. For the *ESP32* processor, we took the minimum

[52] nopnop2002. *esp-idf-benchmark*. Page consultée le 14 Juillet 2024. URL: https://github.com/nopnop2002/esp-idf-benchmark.

and maximum power consumption during normal operation ($P_{min} = U \cdot I_{min} = 3.3 \cdot 0.095 = 0.31W$, $P_{max} = U \cdot I_{max} = 3.3 \cdot 0.240 = 0.79W$[53]). Since the data bus of our processor is 64 bits, and the last bit is reserved for the VRAM, we computed a maximum amount of RAM of 2^{63} bytes.

To compute how many clock cycles are needed to store a 64-bit number, we relied on the documentation of each processor. For the *STR71xF* processor, at least 2 clock cycles are needed to store a 64-bit number, since two 32-bit registers would have to be stored. For the *ESP32* processor, whose registers are 16 bits, at least 4 clock cycles would be needed to store a 64-bit number, since four registers would have to be stored. For our processor, only a single clock cycle is needed to store a 64-bit number, since the registers are 64 bits.

To compute how many clock cycles are needed to add two 64-bit numbers, we used the same method. For the *STR71xF* processor, at least 2 clock cycles are needed to add two 64-bit numbers, since it would be necessary to add 32 bits at a time and add the carries. For the *ESP32* processor, at least 4 clock cycles would be needed to add two 64-bit numbers, since it would be necessary to add 16 bits at a time and add the carries. For our processor, only a single clock cycle is needed to add two 64-bit numbers, since the resolution of the registers is 64 bits.

8.4 Analysis of the results

As we can see, our processor consumes more power than the *STR71xF* processor, but less than the *ESP32* processor. However, since our processor is built on an FPGA, it has a much lower clock speed than the other processors. If it were built directly on a chip, it could have a higher clock speed, and therefore greater performance, since this would also increase the number of instructions processed per second by our processor. Note that, since it has a 64-bit resolution, it takes up more room on the FPGA, and therefore cannot have a clock speed as high as the other processors, whose resolution is 32 bits. Since our processor is very simple, it does not stall to execute complex instructions, and therefore it can process more instructions per clock cycle than the other processors (MIPS/MHz). Finally, our processor has a larger maximum amount of RAM than the other processors, since the resolution is 64 bits.

[53] Khaled Magdy. *ESP32 Sleep Modes & Power Consumption in Each Mode*. Page consultée le 14 Juillet 2024. URL: https://deepbluembedded.com/esp32-sleep-modes-power-consumption/.

9 Conclusion

9.1 Recap

This project enabled us to acquire a great deal of new knowledge, notably the design of microprocessor architectures, the various aspects of implementing their key elements, and the creation of an integral system capable of running various software, including software that uses external peripherals such as a video output or a speaker. We also became familiar with FPGA technology, as well as the tools for simulating and programming them. It was also necessary to deepen the knowledge related to hardware description languages (Verilog). We also developed tools to make it easier for us to translate the assembly in which we wrote, among other things, Tetris, so that we could then translate it into machine code.

Point by point, after the creation of the architecture of our microprocessor, the model in Verilog was developed, simulated, and tested on the FPGA. In addition, by adding several modules to our processor, we arrived at a system capable of running complex and interactive software (notably the game *Tetris*). Then, the platform was integrated into the FPGA using the *Vivado* software. This allowed us to have a functional, high-performance, and versatile gaming computer. To program *Tetris* on it, we first wrote the simulator for this game in C in order to study the logic of this game more closely. Then, we translated the algorithm into the assembly of our own processor while optimizing it to guarantee the speed of its operation on our system. Note also that this required the creation of an assembly compiler (written in Python) that translates it into machine code. Finally, we ran Tetris on our system with sound.

We therefore succeeded in creating a complete computer, capable of running Tetris accompanied by sound effects, with a power consumption as low as $0.311W$ and a clock frequency as slow as $40MHz$. This shows us that by optimizing programs, games can be run even with slow processors in real time.

9.2 Critique

We first implemented the processor. Then, we tested it successfully. We then wrote an assembly compiler in order to be able to implement Tetris. After programming Tetris, we equipped the processor with PWM outputs in order to play sound. This approach turned out to be correct, since it allowed us to reach our goal.

During our approach, at the time of adding GPIO outputs, we could have anticipated the addition of PWM outputs for sound, to do it at the same time.

We could have implemented interrupts, which would allow the processor not to constantly scan the peripherals, but to be notified upon a change (for example, a button press). We did not implement them because we were limited by time. We could also have implemented timers (which are present in almost all other processors), which would allow an operating system to alternate tasks and to know precisely what time it is.

In addition, most processors are equipped with a signal named *WAIT*, which stops the processor. This signal is used in multi-core systems (several processors put together) by the memory to tell the cores of the microprocessor that it is not ready to be used. We also did not implement a memory-access "arbiter" so that the memory could be handled by a multi-core processor. We could have implemented support for having multiple cores, but we did not do so since there are many processors that are not multi-core.

The implementation of an external clock would have been beneficial, to clock the processor and thus its sound sampling rate, to avoid frequency oscillation or other problems.

We could have given more registers to our processor, for example 32. Nowadays, some processors (such as MIPS[54]) have 32 registers. However, we managed to run Tetris on it with 16. Processors with an x86 architecture are only equipped with 8 registers.[55]

Also, we could have added multiplication and division instructions. Adding such instructions would have made our task easier when multiplying by 10. When developing more complicated programs, it would also make the task easier for the programmer. However, adding such instructions would require adding a pipeline, since a multiplication and a division will need several clock cycles to compute the result. We did not implement it in order not to implement a pipeline for computations, so as not to complicate our processor further.

We could also have added operations on floating-point numbers (decimal numbers). This would further facilitate scientific computation, but it would require the addition of a floating-point unit with a pipeline (FPU, *floating point unit*) that would handle the computation of operations concerning these numbers. Being complex, this unit will take up a lot of room on the FPGA and will slow down the processor. We did not implement it so that our processor would remain simple.

If we had developed a C compiler, the programming time for Tetris would have been shorter. However, creating a C compiler takes a lot of time and is very complex, which is why this game was written entirely

[54]Chen, Novick, and Shimano, *MIPS*, op. cit.
[55]Microsoft, *x86 Architecture*, op. cit.

in assembly. We also made this choice to be able to further optimize the system. For example, with a sound implementation written in C, it would be difficult to clock the playback of the sound, since C does not directly give access to the number of clock cycles used, which we had used during our programming of the game in assembly to clock the delays precisely.

9.3 Going further

We could develop a commercial system on a chip (i.e., a complete system embedded on a single integrated circuit, in English *system on a chip* or SoC) from our microprocessor, which could be the basis of a laptop or a tablet with very low power consumption. We could also design a complete operating system, optimized to enable the use of a laptop based on our microprocessor, powered by solar panels. If this project succeeds, a computer of this kind could not only be a formidable competitor to those available on the current market, but also become a precious educational tool for the most disadvantaged populations.

10 Acknowledgements

We would like to thank our Travail de Maturité supervisor, Thomas Morel.
We would also like to thank our expert, René Beuchat, for his invaluable help.
We also thank our parents for teaching us programming from a very young age.

11 Bibliography

Abdelrazek, Ahmed Fathy Mohammed. *Exception and Interrupt Handling in ARM*. Page consultée le 27 Mai 2024. URL: http://classweb.ece.umd.edu/enee447.S2016/ARM-Documentation/ARM-Interrupts-3.pdf.

Adafruit. *40-pin TFT Friend - FPC Breakout with LED Backlight Driver*. Page consultée le 15 Juillet 2024. URL: https://adafru.it/1932.

— *7.0" 40-pin TFT Display - 800x480 without Touchscreen*. Page consultée le 15 Juillet 2024. URL: https://adafru.it/2353.

— *Mini LED Arcade Button - 24mm Translucent Clear*. Page consultée le 15 Juillet 2024. URL: https://adafru.it/3429.

Advanced Micro Devices, Inc. *Adaptive SoCs and FPGAs*. Page consultée le 31 Mai 2024. URL: https://www.amd.com/en/products/adaptive-socs-and-fpgas.html.

— *AMD Artix™ 7 FPGAs*. Page consultée le 31 Mai 2024. URL: https://www.amd.com/en/products/adaptive-socs-and-fpgas/fpga/artix-7.html.

— *Vivado Overview*. Page consultée le 31 Mai 2024. URL: https://www.xilinx.com/products/design-tools/vivado.html.

Alan Bradley, PC MAG. *What Is VRAM? Why Games Are Using it More, and How to Optimize Your PC*. Page consultée le 3 Juin 2024. URL: https://www.pcmag.com/how-to/what-is-vram-why-games-are-using-it-more-and-how-to-optimize-your-pc.

Arduino S.r.l. *Arduino Uno Rev3*. Page consultée le 26 Mai 2024. URL: https://store.arduino.cc/products/arduino-uno-rev3.

ARM Limited. *ARM7TDMI-S Technical Reference Manual*. Page consultée le 13 Juillet 2024. URL: https://documentation-service.arm.com/static/5e8e13a9fd977155116a3368.

Arm Limited. *CPU Architecture*. Page consultée le 1er Juin 2024. URL: https://www.arm.com/architecture/cpu.

Atmel Corporation. *8-bit Atmel Microcontroller with 128KBytes In-System Programmable Flash*. Page consultée le 27 Mai 2024. URL: https://ww1.microchip.com/downloads/en/DeviceDoc/doc2467.pdf.

— *8-bit AVR Microcontroller with 32K Bytes In-System Programmable Flash*. Page consultée le 26 Mai 2024. URL: http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.

Brian Bailey - semiengineering.com. *What Will That Chip Cost?* Page consultée le 10 Juillet 2024. URL: https://semiengineering.com/what-will-that-chip-cost/.

Cassel, David. *Which Programming Languages Use the Least Electricity?* Page consultée le 27 Mars 2024. URL: https://thenewstack.io/which-programming-languages-use-the-least-electricity/.

Chen, Crystal, Greg Novick, and Kirk Shimano. *How Pipelining Works*. Page consultée le 27 Mai 2024. URL: https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/pipelining/index.html.

— *MIPS*. Page consultée le 10 Juillet 2024. URL: https://cs.stanford.edu/people/eroberts/courses/soco/projects/risc/mips/index.html.

codecademy.com. *Compiling*. Page consultée le 23 Juin 2024. URL: https://www.codecademy.com/resources/docs/c/compiling.

Davis, Jim. *HEARING*. Page consultée le 12 Juillet 2024. URL: hhttps://iu.pressbooks.pub/humanphys/chapter/hearing/.

Diferr.com. *10 Differences Between verilog and vhdl*. Page consultée le 26 Mai 2024. URL: https://diferr.com/difference-between-verilog-and-vhdl/.

Digilent. *Cmod A7-35T: Breadboardable Artix-7 FPGA Module*. Page consultée le 31 Mai 2024. URL: https://digilent.com/shop/cmod-a7-35t-breadboardable-artix-7-fpga-module/.

— *Digilent – Start Smart, Build Brilliant*. Page consultée le 1er Juin 2024. URL: https://digilent.com/.

Espressif Systems (Shanghai) Co., Ltd. *ESP32 Technical Reference Manual*. Page consultée le 14 Juillet 2024. URL: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.

Fog, Agner. *Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD, and VIA CPUs*. Page consultée le 27 Mai 2024. URL: https://www.agner.org/optimize/instruction_tables.pdf.

Intel Corporation. *Moore's Law - The past, present and future of Gordon Moore's golden rule for the semiconductor industry. [En Ligne]*. Page consultée le 17 Mars 2024. URL: https://www.intel.com/content/www/us/en/newsroom/resources/moores-law.html.

Jean-Michel Muller et al. *Handbook of Floating-Point Arithmetic*. Page consultée le 27 Mai 2024. URL: https://ieeemilestones.ethw.org/w/images/5/54/Handbook_Floating_Point_Chapter_3.pdf.

Khaled Magdy. *ESP32 Sleep Modes & Power Consumption in Each Mode*. Page consultée le 14 Juillet 2024. URL: https://deepbluembedded.com/esp32-sleep-modes-power-consumption/.

Khan, Muhammad Umair et al. *Measuring power consumption in mobile devices for energy sustainable app development: A comparative study and challenges*. Page consultée le 27 Mars 2024. URL: https://www.sciencedirect.com/science/article/abs/pii/S2210537921000780.

Lenovo. *ALU: Arithmetic Logic Unit Key Components & Logic Gates*. Page consultée le 7 Avril 2024. URL: https://www.lenovo.com/ph/en/glossary/arithmetic-logic-unit/.

libsdl-org. *Simple DirectMedia Layer (SDL) Version 3.0*. Page consultée le 23 Juin 2024. URL: https://github.com/libsdl-org/SDL.

Microsoft. *x86 Architecture*. Page consultée le 1er Juin 2024. URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/x86-architecture.

Mouser Electronics, Inc. *GF0876 CUI Devices*. Page consultée le 15 Juillet 2024. URL: https://www.mouser.ch/ProductDetail/CUI-Devices/GF0876?qs=WyjlAZoYn5229XpWUmpA%252Bg%3D%3D.

NIDEC Group. *Calculating Infinity: The Paradox Of Moore's Law [En Ligne]*. Page consultée le 26 Mars 2024. URL: https://acim.nidec.com/drives/control-techniques/news-and-media/blog/insights/calculating-infinity-the-paradox-of-moores-law.

nopnop2002. *esp-idf-benchmark*. Page consultée le 14 Juillet 2024. URL: https://github.com/nopnop2002/esp-idf-benchmark.

Ozgur, Ayfer. *Lectures 9-10: Sampling Theorem*. Page consultée le 12 Juillet 2024. URL: https://web.stanford.edu/class/engr76/lectures/lecture9-10.pdf.

PCWorld - IDG Communications, Inc. *Shopping for an older CPU? Don't go past this—or you'll regret it*. Page consultée le 27 Mars 2024. URL: https://www.pcworld.com/article/539090/shopping-for-an-older-cpu-dont-go-past-this-or-youll-regret-it.html.

Pratt, Vaughan. *Anatomy of the Pentium Bug*. Page consultée le 10 Juillet 2024. URL: http://boole.stanford.edu/pub/anapent.pdf.

SAMSUNG. *One UI Beta Program - Experience the latest One UI with Android 14*. Page consultée le 27 Mars 2024. URL: https://developer.samsung.com/one-ui-beta.

— *Samsung Galaxy S24 Ultra*. Page consultée le 27 Mars 2024. URL: https://www.samsung.com/us/smartphones/galaxy-s24-ultra/.

Semenov Sergei - histrf.ru. *How a Soviet programmer made the worlds most popular game*. Page consultée le 23 Juin 2024. *Titre traduit*. URL: https://histrf.ru/read/articles/skaz-pro-to-kak-sovetskiy-programmist-samuyu-populyarnuyu-igru-v-mire-izobrel.

STMicroelectronics. *STR71xF*. Page consultée le 13 Juillet 2024. URL: https://media.digikey.com/pdf/Data%20Sheets/ST%20Microelectronics%20PDFS/STR71xF.pdf.

Tennberg, Patrik. *The lost art of code optimization*. Page consultée le 27 Mars 2024. URL: https://www.linkedin.com/pulse/lost-art-optimization-patrik-tennberg.

The Conversation Media Group Ltd, Arnab Hazari. *The future of electronics is light*. Page consultée le 27 Mars 2024. URL: https://theconversation.com/the-future-of-electronics-is-light-68903.

Volk, Andrew M., Peter A. Stoll, and Paul Metrovich. *Recollections of Early Chip Development at Intel*. Page consultée le 26 Mai 2024. URL: https://deramp.com/downloads/mfe_archive/050-Component%20Specifications/Intel/Recollections%20of%20Early%20Chip%20Dev.pdf.

www.makeuseof.com, Ayush Jalan. *Why Do So Many Developers Launch Their Apps on iOS Before Android?* Page consultée le 27 Mars 2024. URL: https://www.makeuseof.com/why-are-cross-platform-apps-released-on-ios-first/.

www.makeuseof.com, Joel Lee. *30 Years Of Tetris: A History Of The World's Most Beloved Puzzle Game*. Page consultée le 27 Mars 2024. URL: https://www.makeuseof.com/tag/30-years-tetris-history-worlds-beloved-puzzle-game/.

Zilog, Inc. *Z80 CPU*. Page consultée le 26 Mai 2024. URL: https://www.zilog.com/docs/z80/um0080.pdf.

Gymnase de Nyon

Travail de Maturité 2024

How to Create and Optimize a Microprocessor-Based System for a Video Game Like Tetris

ADRIAN VLASOV

Class: 3M02

Appendices

Pages 33-159

Supervisor: Mr. Thomas Morel

Expert: Mr. René Beuchat

February 2024

12 Appendices

12.1 Implementation details

For the instructions to be written into RAM at processor startup, to then be read from RAM, we made use of the `initial` keyword and the `$readmemh(..., <register>)` command of Verilog. To access the VRAM, for example to color a pixel of the screen white, we chose that the 64ème bit of the memory address would define the multiplexing between the RAM and the VRAM. If the 64ème bit of the address is set to 1, then this designates the VRAM; otherwise, the RAM. We also implemented access to peripherals (i.e., *GPIO*, which is the acronym for *General-Purpose Input-Output*, of which we only have 16 free, since the screen is connected). When the 10ème bit of the memory address is 1, then the first bit corresponds to button A, the second bit corresponds to button B, and the third bit corresponds to LED A of the board. The remaining 5 bits are set to 0. The following byte, i.e., the second, corresponds to the 8 incoming *GPIO* pins. Byte number 3 corresponds to the 8 outgoing *GPIO* pins. The last 40 bits are set to 0 otherwise. Buttons A and B, as well as LED A, correspond to the buttons and LEDs already soldered onto the Cmod A7 board.

12.2 Problems encountered

If the instruction address, *instr_addr*, is too large, since the instructions and the data are contained in the same memory, the processor will read data as if it were instructions, which will result in unexpected behavior.

12.3 Format of the instruction set

We will explain our instruction set. But first, to read the instruction set, the format is as follows:

Rd denotes the 4-bit address of the destination register, which we modify

Rs denotes the 4-bit address of a source register, which we wish to read

Md indicates the 4-bit address of the register that contains the 64-bit destination address of the external memory, i.e., the address at which we want to write in memory

Ms denotes the 4-bit address of the register that contains the 64-bit source address of the external memory, i.e., the address at which we want to read from memory

***Id**$_{xxx}$* indicates immediate data, i.e., data directly present in the instruction, in *xxx* bits. We will sometimes refer to this notion as a *literal value*.

***Id**$_{hw}$* indicates 32-bit immediate data. "hw" means *half word*, which denotes half of a processor word. In our case, a processor word corresponds to 64 bits, which is why an "hw" has a length of 32 bits. We will sometimes refer to this notion as a *literal value*.

s has a length of 8 bits, and tells the memory unit which bytes to modify. A 1 at position x tells the memory unit to touch byte number x, while a 0 says to do nothing. For example, an s of 00000011 (Figure 30) would tell the memory unit to replace only the first two bytes of the data at a given address.

7	6	5	4	3	2	1	0
0	0	0	0	0	0	1	1
Byte 7	Byte 6	Byte 5	Byte 4	Byte 3	Byte 2	Byte 1	Byte 0

Figure 30: *Representation of the example $s = 00000011$. The untouched bytes are colored in gray.*

c denotes a single control bit, the use of which will be specified in the instruction set below

f which has a length of 4 bits (Figure 31) is a *flag mask*, which shows which flags should be modified by the ALU at the time of executing the instruction. The first bit (counted as 0001) indicates the modification of the *Overflow* flag. The second bit (0010) indicates authorization to modify the *Carry* flag. The third bit allows the ALU to change the *Negative* flag, and finally the fourth bit lets the ALU modify the *Zero* flag.

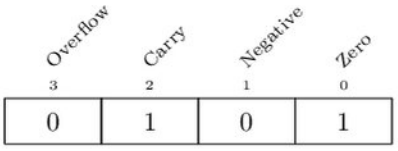


Figure 31: *Example of an $f = 0101$*

$f_{x\ Z,N,C,O}$ is similar to f, except that only the last x bits of the 4 bits will indicate a change. For example, since our flags are ordered as Z, N, C, O, an $f_{2\ Z,N}$ would designate 1100.

fc denotes the desired combination of flags for a few controls. To do this, here is the list of values it can take and their explanations:

- **0 or *000*** means *not equal*, or when the flag $Z = 0$
- **1 or *001*** denotes *equal*, or when the flag $Z = 1$. This is true because, to compare values, we perform a subtraction. When they are equal, the *Zero* flag is set.
- **2 or *010*** denotes *greater than or equal*, or when the flag $C = 0$. This is true because during a subtraction of two numbers, if the first was greater than the second, there would be no overflow.
- **3 or *011*** means *strictly less*, or when the flag $C = 1$.
- **4 or *100*** denotes *strictly greater*, or when $Z = C = 0$. Same explanation as for *greater than or equal*, except that we exclude equality, which is why $Z = 0$.
- **5 or *101*** means *less than or equal*, or when $C = 1$ or $Z = 1$.
- **6 or *110*** denotes *positive*, or when $N = 0$, i.e., when the number is not negative.
- **7 or *111*** denotes *negative*, or when $N = 1$, i.e., when the result is negative.

12.4 Bit fields

The instructions of the processor are encoded in binary form. This is why, to illustrate each instruction format, we have represented below the different possible bit-field formats.

1. **Normal arithmetic instruction** An instruction of this type would be, for example, *addition* or *subtraction*. The example (below the bit field) consists of the *add* instruction redefining $R_5 := R_5 + R_0$ and modifying all the flags.

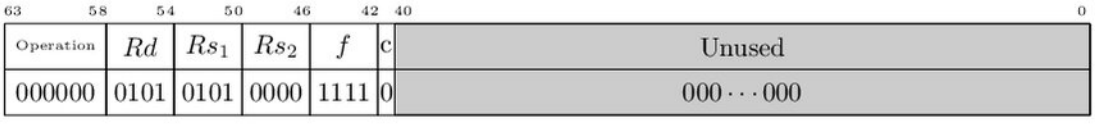


Figure 32: *Bit field of a normal arithmetic instruction with an example below.*

2. **Arithmetic instruction with a literal value** Instructions of this type are, for example, *addi* and *subi*. The example (below the bit field) consists of the *addi* instruction adding $1'234'567'890$ to R_0 and, in doing so, modifying all the flags.

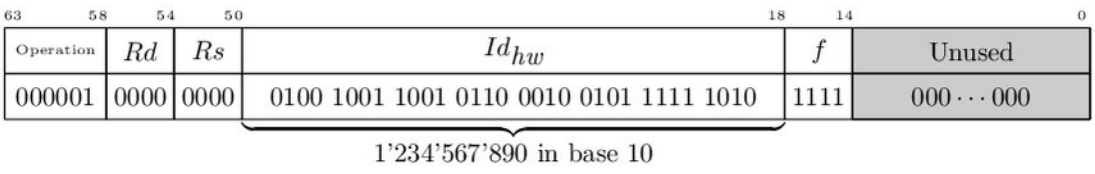


Figure 33: *Bit field of an arithmetic instruction with a literal value, followed by an example.*

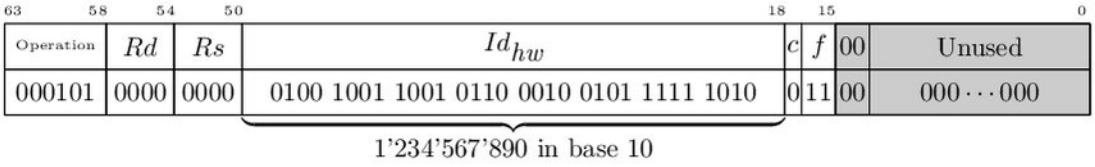
3. **Normal logic instruction** For example, logical AND and logical OR use this format. The last two bits of $f_{2\ Z,N}$ are unmodifiable. The example consists of the *and* instruction redefining $R_5 := R_5|R_0$ and modifying only the Z and N flags.



63	58	54	50	46	42	0
Operation	Rd	Rs_1	Rs_2	f	00	Unused
000101	0001	0001	0000	11	00	$000\cdots000$

Figure 34: *Bit field of a normal logic instruction with an example below.*

4. **Logic instruction with a literal value** For example, *andi* and *ori* use this format. The example consists of the *andi* instruction performing the *AND* operation between $1'234'567'890$ and R_0 and putting the result in R_0. This instruction modifies only the Z and N flags.



63	58	54	50	18	15	0
Operation	Rd	Rs	Id_{hw}	c	f 00	Unused
000101	0000	0000	0100 1001 1001 0110 0010 0101 1111 1010	0	11 00	$000\cdots000$

1'234'567'890 in base 10

Figure 35: *Bit field of a logic instruction with a literal value, followed by an example.*

5. **Single-register instruction modifying 2 flags** For example, *neg* and *rrc* are of this type. The example consists of the *neg* instruction, which takes the two's complement of R_0. It modifies the Z and N flags.



63	58	54	50	0
Operation	Rd	f	00	Unused
001010	0000	11	00	$000\cdots000$

Figure 36: *Bit field of a single-register instruction modifying only 2 flags, followed by an example.*

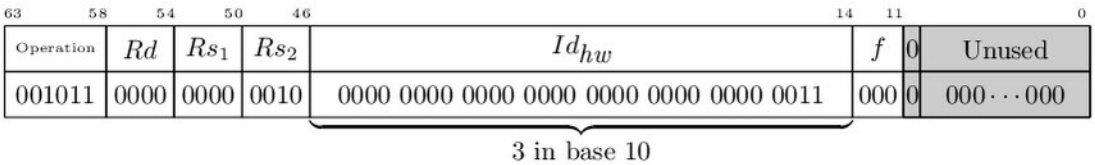
6. **Single-register instruction modifying 3 flags** For example, *asr* is of this type. The example consists of the *asr* instruction, which shifts R_0 to the right while leaving the $64^{\text{ème}}$ bit in place. It modifies the Z and N flags, but not C, due to the 0 in the first position of $f_{3\ Z,N,C}$.



63	58	54	51	0
Operation	Rd	f	0	Unused
001010	0000	110	0	$000\cdots000$

Figure 37: *Bit field of a single-register instruction modifying only 3 flags, followed by an example.*

7. **Logical shift instruction modifying 3 flags** For example, *lsr* is of this type. The example consists of the *lsr* instruction, which shifts R_0 to the right by R_2+3. This instruction, shown as an example, modifies no flags, because we set f to 000.



63	58	54	50	46	14	11	0
Operation	Rd	Rs_1	Rs_2	Id_{hw}	f	0	Unused
001011	0000	0000	0010	0000 0000 0000 0000 0000 0000 0000 0011	000	0	$000\cdots000$

3 in base 10

Figure 38: *Bit field of a logical shift instruction modifying only 3 flags, followed by an example.*

8. **Logical shift instruction with a literal value modifying 3 flags** For example, *lsri* is of this type. The example consists of the *lsri* instruction, which shifts R_0 to the right by 3. This instruction, shown as an example, modifies no flags, because we set f to 000.

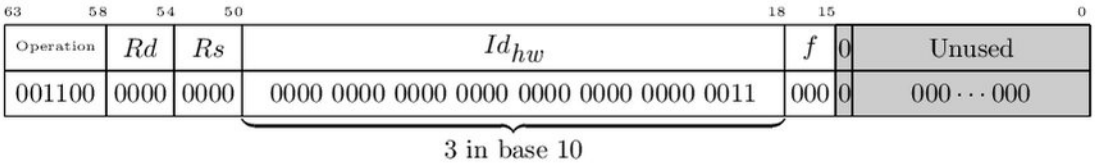


Figure 39: *Bit field of a logical shift instruction with a literal value modifying only 3 flags, followed by an example.*

9. **Two-register instruction** For example, the *cp* instruction is of this type. To illustrate, we have added an example consisting of the *cp* instruction comparing R_1 and R_0.



Figure 40: *Bit field of a two-register instruction, followed by an example.*

10. **Instruction with one register and one literal value** For example, *cpi* uses this bit field. To illustrate, we have added an example consisting of the *cpi* instruction comparing R_0 and 42.

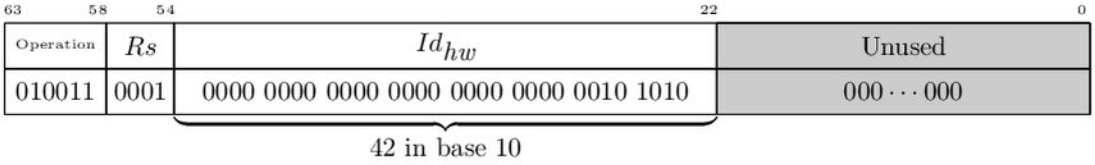


Figure 41: *Bit field of an instruction with one register and one literal value, followed by an example.*

11. **Flags-only instruction** For example, *fst* is of this type. To illustrate, we have added an example consisting of the *fst* instruction which, when executed, sets the Z and C flags to 1.

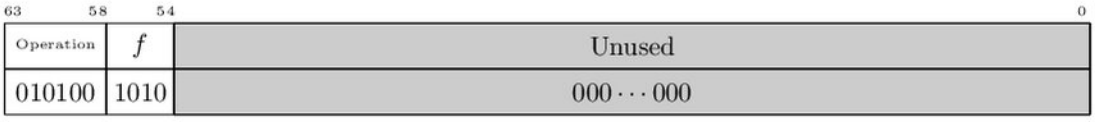


Figure 42: *Bit field of a flags-only instruction, followed by an example.*

12. **Instruction with one literal value** For example, *jmpi* is an instruction of this type. The example consists of the *jmpi* instruction, which forces the processor to jump to address 42.

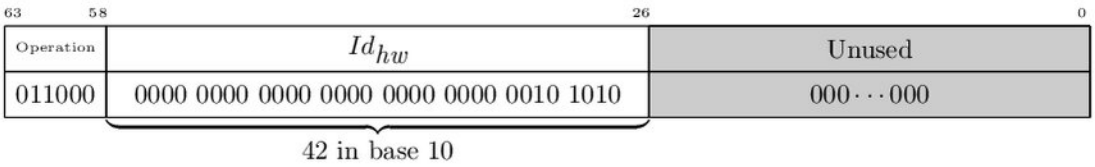


Figure 43: *Bit field of an instruction with one literal value, followed by an example.*

13. Conditional branch instruction with one register and a 29-bit literal value For example, *bif* is of this type. The example consists of the *bif* instruction, which forces the processor to jump to address $R_0 + 42$ if a comparison executed beforehand (such as *cp*, for example) would have set the C or Z flags, i.e., if, for the arguments of this comparison, the relation was *less than or equal*.

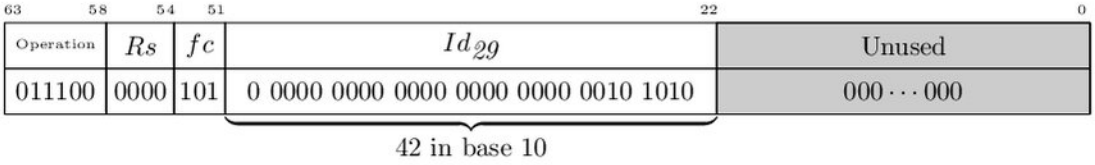


Figure 44: *Bit field of a conditional branch instruction with one register and a 29-bit literal value, followed by an example.*

14. Conditional branch instruction with a 29-bit literal value For example, *bifr* is of this type. The example consists of the *bifi* instruction, which forces the processor to jump to address 42 if a comparison executed beforehand (such as *cp*, for example) would have set the C flag to 0, i.e., for the arguments of this comparison, the relation was *greater than or equal*.

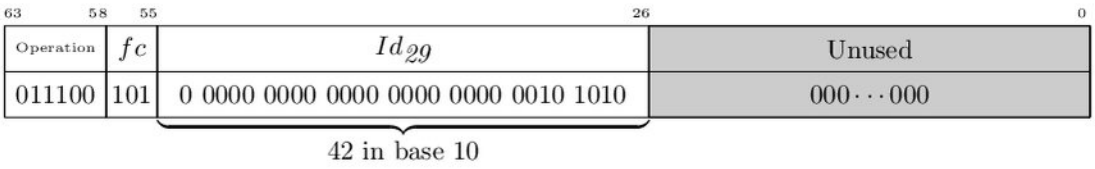


Figure 45: *Bit field of a conditional branch instruction with a 29-bit literal value, followed by an example.*

15. Instruction with two registers, one literal value, and an 8-bit mask For example, *mov* is of this type. The example consists of the *mem* instruction, which puts the data contained in R_1 at the address contained in R_0, where we decided to write only the odd bytes (1, 3, 5, and 7).

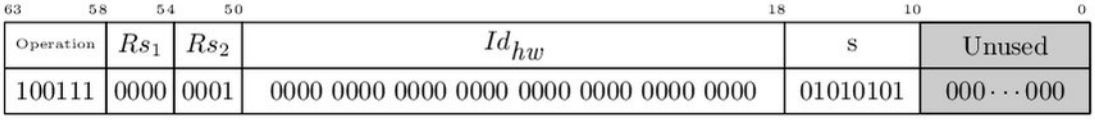


Figure 46: *Bit field of an instruction with two registers, one literal value, and an 8-bit mask, followed by an example.*

16. Instruction with one register, one literal value, and an 8-bit mask For example, *memi* uses this bit field. The example consists of the *memi* instruction, which puts the number 42 at the address contained in R_0, where we decided to write all the bits.

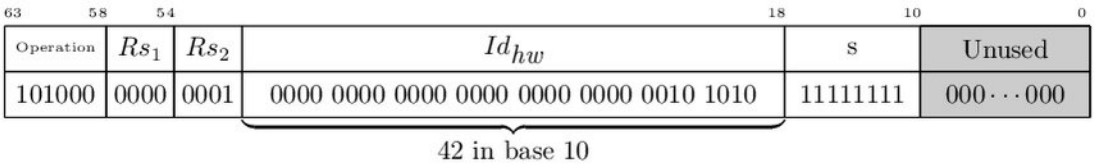


Figure 47: *Bit field of an instruction with one register, one literal value, and an 8-bit mask, followed by an example.*

17. Instruction with one register, one literal value, and one control bit For example, *ldi* uses this bit field. The example consists of the *ldi* instruction, which initializes the first 32 bits of R_0 to the number 42.

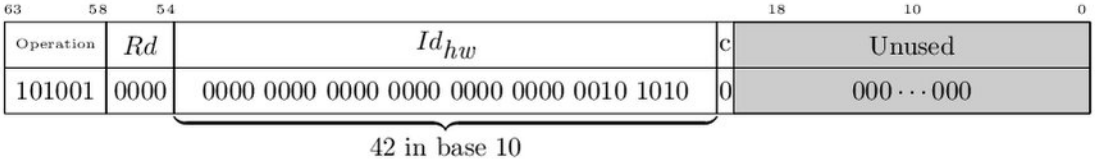


Figure 48: *Bit field of an instruction with one register, one literal value, and one control bit, followed by an example.*

18. Single-register instruction For example, *sas* is of this type. The example consists of the *sas* instruction, which saves the address of a subroutine in register R_0.

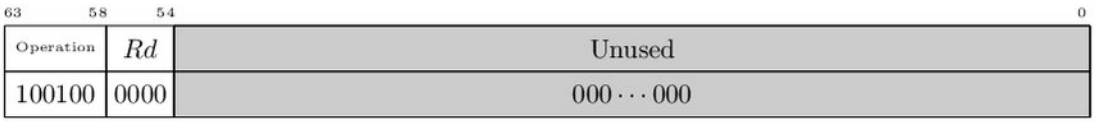


Figure 49: *Bit field of a single-register instruction, followed by an example.*

19. The *nop* instruction The bit field for the *nop* instruction is shown below (Figure 50). This instruction does strictly nothing, which is why it has no operands. Numbers 37 and above can be used to represent *nop*.

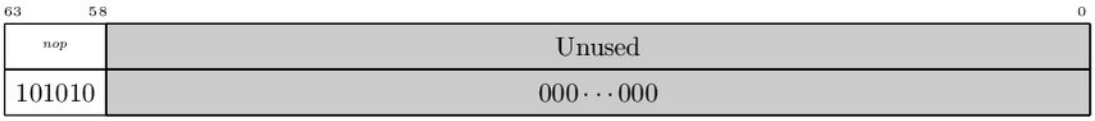


Figure 50: *Bit field of the nop instruction, followed by an example.*

12.5 Instruction set

Here is the instruction set of our processor, where each number will correspond to the real instruction number, and where the type of bit field will be mentioned alongside:

Format: Instruction	Bit field
0. ***add*** Rd, Rs_1, Rs_2, f, c	1
1. ***addi*** Rd, Rs, Id_{hw}, f	2
2. ***sub*** Rd, Rs_1, Rs_2, f, c	1
3. ***subi*** Rd, Rs, Id_{hw}, f	2
4. ***and*** Rd, Rs_1, Rs_2, $f_{2\ Z,N}$	3
5. ***andi*** Rd, Rs, Id_{hw}, c, $f_{2\ Z,N}$	4
6. ***or*** Rd, Rs_1, Rs_2, $f_{2\ Z,N}$	3
7. ***ori*** Rd, Rs, Id_{hw}, c, $f_{2\ Z,N}$	4

8. ***xor*** Rd, Rs_1, Rs_2, $f_{2\ Z,N}$ 3

9. ***xori*** Rd, Rs, Id_{hw}, c, $f_{2\ Z,N}$ 4

10. ***neg*** Rd, $f_{2\ Z,N}$ 5

11. ***lsr*** Rd, Rs_1, Rs_2, Id_{hw}, $f_{3\ Z,N,C}$ 7

12. ***lsri*** Rd, Rs, Id_{hw}, $f_{3\ Z,N,C}$ 8

13. ***lsl*** Rd, Rs_1, Rs_2, Id_{hw}, $f_{3\ Z,N,C}$ 7

14. ***lsli*** Rd, Rs, Id_{hw}, $f_{3\ Z,N,C}$ 8

15. ***rrc*** Rd, $f_{2\ Z,N}$ 5

16. ***rlc*** Rd, $f_{2\ Z,N}$ 5

17. ***asr*** Rd, $f_{3\ Z,N,C}$ 6

18. ***cp*** Rs_1, Rs_2 9

19. ***cpi*** Rd, Id_{hw} 10

20. ***fst*** f 11

21. ***fcl*** f 11

22. ***jmp*** Rs, Id_{hw} 10

23. ***jmpr*** Id_{hw} 12

24. ***jmpi*** Id_{hw} 12

25. ***cs*** Rs, Id_{hw} 10

26. ***csr*** Id_{hw} 12

27. ***csi*** Id_{hw} 12

28. ***bif*** Rs, fc, Id_{29} 13

29. ***bifr*** fc, Id_{29} 14

30. ***bifi*** fc, Id_{29} 14

31. ***bio*** Rs, Id_{hw} 10

32. ***bior*** Id_{hw} 12

33. ***bioi*** Id_{hw} 12

34. ***ret*** Id_{hw} 12

35. ***rti*** was to be the interrupt-return command; we decided to do without it

36. ***sas*** Rd 18

37. ***sal*** Rs 18

38. ***mov*** Rd, Ms, Id_{hw}, s 15

39. ***mem*** Md, Rs, Id_{hw}, s 15

40. ***memi*** Md, Id_{hw}, s 16

41. ***ldi*** Rd, Id_{hw}, c 17

42 or more. ***nop*** 19

Here are the explanations for each instruction:

0: ***add*** *Rd*, Rs_1, Rs_2, **f, c** adds Rs_1 and Rs_2 together and puts the result in *Rd*. The programmer can choose which flags to authorize with *f*. The control bit *c*, when set, uses the previous overflow, if there is one, to perform the addition. This is useful when adding numbers that need more room to be stored (for example, 128 bits). The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

1: ***addi*** *Rd*, *Rs*, Id_{hw}, **f** adds *Rs* and the number in Id_{hw} together and puts the result in *Rd*. The programmer can also choose which flags to authorize with *f*. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

2: ***sub*** *Rd*, Rs_1, Rs_2, **f, c** subtracts Rs_2 from Rs_1 and puts the result in *Rd*. The programmer can choose which flags to authorize with *f*. The control bit *c*, when set, uses the previous overflow, if there is one, to perform the subtraction. This is useful when subtracting numbers that need more room to be stored (for example, 128 bits). The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

3: ***subi*** *Rd*, *Rs*, Id_{hw}, **f** subtracts Id_{hw} from *Rs* and puts the result in *Rd*. The programmer can also choose the flags to authorize with *f*. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

4: ***and*** *Rd*, Rs_1, Rs_2, $f_{2\ Z,N}$ This command performs the *AND* operation between Rs_1 and Rs_2 and puts the result in *Rd*. This instruction only generates two flags, *Zero* and *Negative*, because to generate *Carry* or *Overflow*, a mathematical operation such as addition would be required. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

5: ***andi*** *Rd*, *Rs*, Id_{hw}, **c,** $f_{2\ Z,N}$ Since there are only 32 bits in Id_{hw}, by using *c*, the programmer could choose to perform the logical AND operation on the first 32 bits or the last 32 bits of the register. When *c* is set to 1, the processor performs this operation on the last 32 bits of register *Rd*; otherwise, on the first 32. This command only generates two flags, *Zero* and *Negative*, for the same reason as *and*.

6: ***or*** *Rd*, Rs_1, Rs_2, $f_{2\ Z,N}$ performs the *OR* operation between Rs_1 and Rs_2 and puts the result in *Rd*. This command only generates two flags, *Zero* and *Negative*, for the same reason as *and*. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

7: ***ori*** *Rd*, *Rs*, Id_{hw}, **c,** $f_{2\ Z,N}$ When *c* is set to 1, the processor performs the *OR* operation on the last 32 bits between register *Rs* and Id_{hw}; otherwise, on the first 32, and puts it in the register at address *Rd*. This instruction only generates two flags, *Zero* and *Negative*.

8: ***xor*** *Rd*, Rs_1, Rs_2, $f_{2\ Z,N}$ performs the *exclusive OR* operation, also called *XOR*, between Rs_1 and Rs_2 and puts the result in *Rd*. This command only generates two flags, *Zero* and *Negative*. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

9: ***xori*** *Rd*, *Rs*, Id_{hw}, **c,** $f_{2\ Z,N}$ When *c* is set to 1, the processor performs the *XOR*, or *exclusive OR*, operation on the last 32 bits between register *Rs* and Id_{hw}; otherwise, on the first 32, and puts it in the register at address *Rd*. This command only generates two flags, *Zero* and *Negative*. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

10: ***neg*** *Rd*, $f_{2\ Z,N}$ takes the two's complement of the number in register *Rd*. This command only generates two flags, *Zero* and *Negative*, for the same reason as *and*. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

11: ***lsr*** *Rd*, Rs_1, Rs_2, Id_{hw}, $f_{3\ Z,N,C}$ shifts the number of Rs_1 to the right $Rs_2 + Id_{hw}$ times, then writes it into *Rd*. This instruction generates *Zero* and *Negative*, like all the ones we have listed so far and for the same reason. *Carry* is also generated, but this time the flag is set to the bit number $Rs_2 + Id_{hw} - 1$ of Rs_1, in order to be able to shift numbers larger than 64 bits. If $Rs_2 + Id_{hw} = 0$, then *Carry* is set to 0. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

12: ***lsri*** *Rd*, *Rs*, Id_{hw}, $f_{3\ Z,N,C}$ shifts the number of *Rs* to the right Id_{hw} times, then writes it into *Rd*. This instruction generates *Zero* and *Negative*. *Carry* is also generated, but this time the flag is set to the bit number $Id_{hw} - 1$ of *Rs*, for the same reason as in *lsr*. If $Id_{hw} = 0$, then *Carry* is

set to 0. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

13: ***lsl*** Rd**,** Rs_1**,** Rs_2**,** $Id_{\boldsymbol{hw}}$**,** $f_{3\ Z,N,C}$ shifts the number of Rs_1 to the left $Rs_2 + Id_{hw}$ times, then writes it into Rd. This instruction generates *Zero* and *Negative*. *Carry* is also generated and is set to the bit number $64 - Rs_2 - Id_{hw}$ of Rs_1, in order to be able to shift numbers larger than 64 bits. If $Rs_2 + Id_{hw} = 0$, then *Carry* is set to 0. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

14: ***lsli*** Rd**,** Rs**,** $Id_{\boldsymbol{hw}}$**,** $f_{3\ Z,N,C}$ shifts the number of Rs to the left Id_{hw} times, then writes it into Rd. This instruction generates *Zero* and *Negative*. *Carry* is generated and takes the value of bit number $64 - Id_{hw}$ of Rs, for the same reason as in *lsl*. If $Id_{hw} = 0$, then *Carry* is set to 0. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

15: ***rrc*** Rd**,** $f_{2\ Z,N}$ shifts Rd to the right by 1 and sets the 64ème bit to the one that was in the *Carry* flag at that precise moment. This instruction also generates *Zero* and *Negative*. The new *Carry* takes the value of bit number 0 of the old Rd. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

16: ***rlc*** Rd**,** $f_{2\ Z,N}$ shifts Rd to the left by 1 and sets the first bit to the one that was in the *Carry* flag at that precise moment. This instruction also generates *Zero* and *Negative*. The new *Carry* takes the value of the 64ème bit of the old Rd. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

17: ***asr*** Rd**,** $f_{3\ Z,N,C}$ shifts Rd to the right by 1, but leaves the 64ème bit in place. Thus the 64ème bit stays at position 64, but is also duplicated at position 63. This instruction generates *Zero* and *Negative*. *Carry* takes the value of the very first bit of Rd before the operation. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

18: ***cp*** Rs_1**,** Rs_2 subtracts Rs_2 from Rs_1, but does not store the result. It only stores the *Zero*, *Negative*, and *Carry* flags. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

19: ***cpi*** Rd**,** $Id_{\boldsymbol{hw}}$ subtracts Id_{hw} from Rs_1, but does not store the result. It only stores the *Zero*, *Negative*, and *Carry* flags. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

20: ***fst*** **f** sets the flags based on f. For example, an f of 0101 would set the *Overflow* and *Negative* flags. The flags will be immediately available at the next clock cycle and can therefore be used by the next instruction.

21: ***fcl*** **f** sets to 0 the flags based on f. For example, an f of 1010 would set the *Zero* and *Carry* flags to 0. The flags will be immediately available at the next clock cycle and can therefore be used by the next instruction.

22: ***jmp*** Rs**,** $Id_{\boldsymbol{hw}}$ sets *PC_NEW* to $Rs + Id_{hw}$. It allows a jump to the address $Rs + Id_{hw}$. Since the processor modifies *PC_NEW*, the jump will not happen at the next clock edge, but after. So it will let only one instruction execute after it before jumping to the specified address.

23: ***jmpr*** $Id_{\boldsymbol{hw}}$ is identical to *jmp*, except that it jumps to the address Id_{hw} + *the current address*, i.e., $Id_{hw} + PC$.

24: ***jmpi*** $Id_{\boldsymbol{hw}}$ is identical to *jmp*, except that it jumps to the address Id_{hw}.

25: ***cs*** Rs**,** $Id_{\boldsymbol{hw}}$ is identical to *jmp*, except that, just before jumping, it saves the current address in an internal register called *subroutine_return_address*, i.e., $PC + 1$, to be able to return to it later. In doing so, it jumps to a "subroutine", i.e., a group of instructions that, after being executed, will return to the starting address. The analogy of a subroutine can be described as an "argument-less function", or a "method".

26: ***csr*** $Id_{\boldsymbol{hw}}$ is identical to *cs*, except that it jumps to the address Id_{hw} + *the current address*, i.e., $Id_{hw} + PC$.

27: ***csi*** $Id_{\boldsymbol{hw}}$ is identical to *cs*, except that it jumps to the address Id_{hw}.

28: ***bif*** Rs**, fc,** Id_{29} is executed if the condition set in fc matches the current flags. If it is executed, then it jumps to the address $Rs + Id_{29}$. It takes as long to jump as *jmp*.

29: ***bifr*** **fc,** Id_{29} is identical to *bif*, except that it jumps to the address Id_{29} + *the current address*, i.e., $Id_{29} + PC$.

30: ***bifi*** **fc,** Id_{29} is identical to *bif*, except that it jumps to the address Id_{29}.

31: ***bio*** Rs**,** $Id_{\boldsymbol{hw}}$ is executed if the *Overflow* flag is set. If it is executed, then it jumps to the address $Rs + Id_{hw}$. It takes as long to jump as *jmp*.

32: ***bior*** $Id_{\boldsymbol{hw}}$ is identical to *bio*, except that it jumps to the address Id_{hw} + *the current address*, i.e., $Id_{hw} + PC$.

33: ***bioi*** $Id_{\boldsymbol{hw}}$ is identical to *bio*, except that it jumps to the address Id_{hw}.

34: ***ret*** $Id_{\boldsymbol{hw}}$ allows returning to the address saved in *subroutine_return_address* by one of the instructions named *csx*.

35: ***rti*** was to be the interrupt-return command; we decided to do without it

36: ***sas*** Rd saves *subroutine_return_address* in register R_d.

37: ***sal*** Rs resets *subroutine_return_address* to the value contained in R_s.

38: ***mov*** Rd**,** Ms**,** $Id_{\boldsymbol{hw}}$**, s** copies 8 bytes of information at the address $Ms+Id_{hw}$ from memory, which it deposits in register Rd. Note that, to copy a byte or not, the respective bit of that byte must be set in s. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

39: ***mem*** Md**,** Rs**,** $Id_{\boldsymbol{hw}}$**, s** copies 8 bytes of information from Rs to $Md + Id_{hw}$ in memory. s has the same effect as in *mov*. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

40. ***memi*** Md**,** $Id_{\boldsymbol{hw}}$**, s** sets 8 bytes in memory to Id_{hw}. If Id_{hw} is negative, i.e., if its 32ème bit is 1, then bits 33 to 64 inclusive are set to 1; otherwise, 0. s has the same effect as in *mov*. The result will be immediately available at the next clock cycle and can therefore be used by the next instruction.

41: ***ldi*** Rd**,** $Id_{\boldsymbol{hw}}$**, c** With this instruction, if c=0, then the first 32 bits of register Rd are set to Id_{hw}. If c=1, then it is the last 32 bits that are set to Id_{hw}. Generally, to set default values in registers, two instructions are therefore needed to do it.

42 and above: ***nop*** does strictly nothing.

12.6 Interview

Here are a few questions we asked our expert.

Adrian: I am thinking of implementing a 64-bit memory bus. Is that too much?
Expert: Since you are looking to make a very simple processor, and you want to optimize it for Tetris rather, you do not need a lot of RAM. Since your FPGA is small, a 64-bit bus will take up all the room in the FPGA, which will not only slow down the processor but also leave no room for the other modules. How much memory are you thinking of using?
Adrian: I am thinking of using 4kb at most.
Expert: So it would be better to implement a 16-bit memory bus, or 32-bit at most, since 16-bit gives access to 64 kilobytes of addressable memory.

Adrian: I am thinking of having only a single interrupt. To know which peripheral activated this interrupt, the processor will ask each connected device whether it was the one.
Expert: This is a good idea. You could also make an interrupt controller external to the processor, which will multiplex each interrupt from each peripheral.

Adrian: Is it better to have a clock, a reset, and an interrupt that trigger on a positive or negative signal? (posedge or negedge)
Expert: Normally, FPGA chips have a posedge Clock and reset. What matters is that you stay consistent throughout your design, so the choice is yours.

Adrian: When I have an interrupt, what should I do with my pipeline?

Expert: When you have an interrupt, you must stop the execution of new instructions and finish your pipeline. After that, your processor will jump to the predefined interrupt address.

Adrian: My instructions are Big Endian, while my registers are Little Endian. In essence, if you count the bits in Little Endian fashion (i.e., from right to left), the instruction's opcode starts at the 63rd bit and ends at the 58th bit inclusive (6 bits, because 64 instructions). Should anything be changed?

Expert: Most processors are Little Endian, so it would be better for the opcode to start at bit 0. But there is not much difference in speed, so the way you have done it will also work.

Adrian: I am thinking of having a dual-port memory (you can read from two different addresses at the same time). It would be simpler to read instructions at the same time as data from the same memory, because I want a Harvard architecture (instructions and data are accessible at the same time, i.e., in parallel).

Expert: Standard memories have only a single port. Nowadays, the majority of processors stop executing new instructions and wait until the processor finishes storing or reading data into memory. So it would be better for your processor to wait too and halt the execution of instructions during the data transfer.

Adrian: To show what my processor can do, I would like to write Tetris as a demo on it. I can program in assembly, but it will take me more time than writing Tetris in C. Which compiler should I adapt if I were to adapt one?

Expert: GCC (Gnu C Compiler) is a good C compiler, but it is very large.

Adrian: I also know Clang, an LLVM compiler, so it generates the same AST (Abstract Syntax Tree, a very basic code). If I adapt the final compiler, the one that transforms the AST into assembly directly, I could compile not only C, but also C++, Rust, Swift, etc. Everything that is LLVM.

Expert: Clang has a different API than GCC, so it is not a compiler that everyone uses and that is not standard. Its standard libraries are different from those of GCC. I think it would be better to adapt GCC than Clang for these reasons.

Adrian: I think the architecture of my microprocessor will resemble that of MIPS or that of ATMega. Should I consider small compilers that already support MIPS, such as LCC (Little C compiler)?

Expert: Yes, you could well base it on that, to save time.

12.7 Source Code and Files

File structure:

```
/
└── src
    ├── processor
    │   ├── ALU.v
    │   ├── constants.v
    │   ├── core.v
    │   ├── display_controller.v
    │   ├── flag_mem.v
    │   ├── flag_mux.v
    │   ├── instruction_decoder.v
    │   ├── io_controller.v
    │   ├── load_store_engine.v
    │   ├── memory_controller.v
    │   ├── program_controller.v
    │   ├── reg_mem.v
    │   ├── reg_mux.v
    │   ├── tb2.v
    │   └── wrapper.v
    ├── tetris-c-algorithm
    │   └── main.cpp
    ├── compiler
    │   └── transpiler.py
    └── asm
        └── tetris.s
```

src/processor/ALU.v